Outside the Prompt: Measuring Where AI Coding Agents Trust Repositories, Tools and Connectors

Paolo Costanzo[1*]

[1*]Independent researcher, CoDe RTD, Turin, Italy.

Corresponding author(s). E-mail(s): me@paolocostanzo.com;

Abstract

Context: AI coding agents run commands and use tools through the Model Context Protocol (MCP). Security work mostly studies prompt injection. **Objective:** Measure where disclosed failures of tool-using agents and connectors trigger relative to the model's decision, and how clients behave at configuration and tool boundaries. **Method:** A preregistered study: classification of 120 stratified advisory-database disclosures of one quarter by the author and three open-weight models; eight clients measured with canary servers; a scored test of default logs; and, exploratory, a trust-dialog study and a census of six official servers. **Results:** In interactive sessions no client started a repository-declared server before the user answered (one non-interactive mode does, as documented), yet five of six clients that ask pre-select the permissive answer. When an approved tool's description changed mid-session over stdio, no client that asked at first call asked again; one fetched it when notified, two on restart. Per server, 0–97% (median 10.7%) of prose-returning tools pass fully through the server's defence. Default logs missed silent tool changes and server starts in two clients each. With class 3 read broadly, the author placed 90.1% of the frame (95% CI 77.9–95.8%) and the model majority 92.6% outside the model's decision (not robust; Fleiss' $\boldsymbol{\kappa = 0.46}$, below the preregistered 0.6); post hoc, blind AI readings of the sources document 63.5% (49.1–74.5%) as such, 33 of 120 unsettled. **Conclusions:** Several security-relevant decisions lie in product defaults and configuration, outside the model. Evaluations should report client, version and configuration, not only the model.

Keywords: AI coding agents, Model Context Protocol, workspace trust, security measurement, preregistration

1 Introduction

Coding agents put a large language model inside a developer tool: a command-line or editor client that reads the repository, runs commands and calls external tools, increasingly through the Model Context Protocol (MCP) (Model Context Protocol 2026a). Most security research on tool-using agents measures whether a model can be induced to follow injected instructions (Wang et al. 2026; Yan et al. 2026; Wu et al. 2026; Zhu et al. 2026). Public disclosures about these tools, however, often describe something else: code that runs while an agent gathers context before the user's trust decision (Rosales 2026), approval dialogs that hide what is about to run (Utevsky 2026a; Luz 2026), and connector endpoints without authentication (Ruflo maintainers 2026). In such cases the outcome is decided by the tool — its defaults, dialogs, configuration loading and plumbing — before or around the model's decision.

For software engineering this matters in two ways. Builders of agent tools control these decisions directly, and evaluations that name only a model leave out the part of the system where many failures occur. We therefore measure, with a preregistered design, how much of the problem sits outside the model in advisory-database disclosures of one quarter (RQ1), and how current products behave at four boundaries that public reports describe: three that the client controls, namely the trust decision for repository-declared servers (RQ2), the approval of a changed tool (RQ3) and what the default logs record (RQ5), and one that the server controls, whether its untrusted-content defence wraps a tool's output (RQ4). These boundaries were chosen from those reports, not from RQ1's result; the first two correspond to RQ1's classes 1 and 2, which are a minority of the sampled disclosures for every rater (8 of 120 author labels and 18 for the model majority; Section 5), so RQ1 does not measure how often they fail. We study five research questions, fixed in a preregistration deposited on 11 October 2026 (Costanzo 2026c):

- **RQ1** (confirmatory) Among public disclosures affecting tool-using AI agents and their connectors between 26 June and 24 September 2026, what share triggers outside the model's decision (classes 1–3) rather than through manipulation of the model (class 4)? H1: the share is at least 60%. H1a: agreement between the raters is at least substantial (Fleiss' $\kappa \geq 0.6$). (Deviation D44 replaced this population with an advisory-database frame; Section 5.)
- **RQ2** (exploratory) Does any current coding-agent client start a repository-declared MCP server before the user's trust decision, and which answer is pre-selected?
- **RQ3** (confirmatory) When an approved tool's definition changes, which clients notify, require re-approval, accept silently or refuse? H3: at most one client in the tested set requires re-approval by default; no client acts on the server's list-changed notification by asking the user.
- **RQ4** (exploratory) In official MCP servers that ship an untrusted-content defence, what share of third-party-content tools route their output through it?
- **RQ5** (confirmatory) Can default host telemetry distinguish each RQ2 and RQ3 event from normal use with a false-positive rate below 5% per hour on a two-hour baseline? H5: at least one event type is not distinguishable with default client logging alone.

Lab names used in the released artifacts map to research questions as follows: Q5 → RQ2, Q3 → RQ3, Q2 → RQ4, Q4 → RQ5. A planned measurement of code that runs before the trust decision (lab Q1) was not carried out (Section 11).

Contributions. (1) A confirmatory classification of a random sample of advisory-database disclosures by trigger point, with the author and three open-weight model raters, frozen criteria, a public sampling seed and published labels (the protocol is preregistered; the operational reading of class 3, which goes beyond its literal wording for 58 of the author's 100 class-3 labels, is audited post hoc, Section 5); since a post hoc reading of the primary sources settles the route for only 87 of the 120 cases, it gives context and a range rather than the main evidence, which comes from (2)–(5); (2) a replication and extension of earlier observations on the trust boundary of repository-declared MCP servers (Utevsky 2026b), with eight clients, exact prompts and defaults, two versions of four of them and a reproducible harness; (3) a confirmatory test of how clients handle a changed tool definition; (4) a tool-by-tool census of untrusted-content defences in official MCP servers; (5) a scored test of what default client logs reveal, with host auditing configured by the lab reported apart; and (6) an open harness, evidence and analysis archived with a DOI (Costanzo 2026a); the version with the RQ1 analysis (v0.5.0) is released with this paper.

What is new. RQ2 is in part a replication: for Gemini CLI and Cursor CLI it confirms observations of TrustFall (Utevsky 2026b) (for non-interactive Claude Code our own record is weak, Section 6), and adds clients, prompt wording, defaults and versions. RQ3 and RQ5 are new measurements: we know of no earlier measurement of whether clients notify or ask again when an approved definition changes at run time, or of whether default client logs separate these events from normal use. RQ4 is a census of six servers, not a sample, and RQ1 is a classification of advisory-database disclosures whose class 3 reading and source support are audited post hoc.

2 Background

Coding agents and workspace trust.

Terminal clients (Claude Code, Codex CLI, Gemini CLI, Qwen Code, Cursor CLI, Goose) and editor clients (Visual Studio Code with GitHub Copilot, Cursor) run a model with access to the file system, a shell and tools. Most ask the user to trust a folder before using it; what that decision covers differs between clients.

The Model Context Protocol.

MCP connects a client to servers that expose tools, prompts and resources (Model Context Protocol 2026a). A project can declare servers in configuration files that the client reads from the repository (`.mcp.json`, `.vscode/mcp.json`, `.cursor/mcp.json`, `.gemini/settings.json`, `.qwen/settings.json`, `.codex/config.toml`); starting such a server runs a command chosen by whoever wrote the repository. Revision 2026-07-28 of the protocol is stateless and has no mechanism for the integrity of tool definitions; a server may announce that its tool list changed (`notifications/tools/list_changed`), and the list may also be cached with a time-to-live after which the client fetches it again without any notification, while the server

may change it in the meantime (Model Context Protocol 2026a,b). A changed tool definition is therefore ordinary protocol behaviour, and what a client does with it is a product decision.

Untrusted-content defences.

Some MCP servers that return third-party text (issue bodies, logs, documents) wrap it in delimiters or markers so that the model can treat it as data rather than instructions, a technique known as spotlighting (Hines et al. 2024). Such spotlighting reduces but does not remove the effect of injected text, especially under adaptive attacks (Nasr et al. 2025); RQ4 measures only whether a server routes a tool's output through its own defence, not whether the defence works.

3 Related work

Prompt injection and tool poisoning.

Indirect prompt injection was first shown on deployed LLM-integrated applications by Greshake et al. (2023), and Liu et al. (2024b) formalise and benchmark prompt-injection attacks and defences. Benchmarks then measured it for tool-using agents (Zhan et al. 2024; Debenedetti et al. 2024), including untrusted tool feedback (Yan et al. 2026). For the Model Context Protocol, MCPTox measures tool poisoning on live servers (Wang et al. 2026), and Liu et al. (2026) build sandboxed tool-description poisoning cases. Recent work extends the model-centred view to coding agents: injection through poisoned development resources in Cursor and GitHub Copilot (Liu et al. 2025), compositions of individually benign commands (Wu et al. 2026), and repository poisoning under different user invocations (Zhu et al. 2026). Adaptive attacks bypass most published defences, including delimiting and detector-based ones (Nasr et al. 2025). Other work moves the security decision out of the model by design: CaMeL separates control flow from untrusted data (Debenedetti et al. 2025), and design patterns limit what an agent can do after reading untrusted input (Beurer-Kellner et al. 2025); these are proposed architectures, whereas RQ2–RQ4 measure the defaults that current clients and servers ship. The attack and benchmark studies above ask whether a *model* follows injected content. We ask instead which outcomes are fixed by the *client* (defaults, dialogs, configuration) before or regardless of the model's decision.

MCP security analyses.

Hou et al. (2025) propose a lifecycle and threat taxonomy for MCP servers. Radosevich and Halloran (2025) show that models can be coerced into misusing MCP tools. Song et al. (2025) evaluate tool poisoning, puppet and rug-pull attacks through malicious servers. Large-scale studies examine server code: Hasan et al. (2026) on security and maintainability, VIPER-MCP on taint-style flaws (Sun et al. 2026), and Toeppe et al. (2026) provide a dataset of implementations. Huang et al. (2026) compare seven MCP clients with recorded versions and models, but they study defences against tool poisoning, not trust dialogs or the start of repository-declared servers. RQ3 relates to the rug pull, which Beurer-Kellner and Fischer (2025) described (a server changes a tool's description after the client approved it) without measuring it; they proposed

pinning tools and hashing their descriptions. Signed, versioned tool definitions (Bhatt et al. 2025) and digest-pinned tool versions (Model Context Protocol 2025) have been proposed since. MCPSecBench (Yang et al. 2025) measured the success of a rug-pull attack in Claude Desktop, OpenAI's client and Cursor, but did not check whether a client notifies the user or asks for approval again. RQ3 does not demonstrate a new attack. It measures how clients react at run time when a tool definition changes, a change the protocol allows (Model Context Protocol 2026b). This differs from a change to a project's MCP configuration after approval (MCPoison, CVE-2025-54136 (Check Point Research 2025)), after which Cursor 1.3 requires approval again.

Trust dialogs and repository-declared servers.

Running what a project declares is a known supply-chain concern: installing a package extends trust to code and maintainers the user has not reviewed (Zimmermann et al. 2019), Ohm et al. (2020) review real malicious packages, including ones that act at installation, and Ladisa et al. (2023) give a taxonomy of attacks on open-source supply chains; an agent configuration file that declares a server is one more project artifact of this kind. Before coding agents, Lin et al. (2024) studied weaknesses in VS Code extensions that handle untrusted workspace input. Client-specific disclosures preceded any cross-client comparison: MCPoison showed that Cursor bound its approval of a project-declared MCP server to the server's name, so a repository could change the approved entry afterwards (Check Point Research 2025). The closest multi-client comparison is the TrustFall report (Utevsky 2026b). It compared Claude Code, Gemini CLI, Cursor CLI and Copilot CLI. It reported that the trust prompt pre-selects trusting in all four CLIs (Claude Code at v2.1.129), that repository-declared MCP servers start once the folder is trusted, and that headless Claude Code shows no dialog. RQ2 replicates these observations for Gemini CLI and Cursor CLI rather than discovering them (for headless Claude Code our own record is weak); for interactive Claude Code on later versions its result differs (Section 10). It covers eight clients, two of them graphical, the exact wording and default of each prompt, several client versions, the non-interactive path on four clients and a public harness.

Classification of disclosures and model raters.

We classify public disclosures by where they trigger. Mining vulnerability advisories of software ecosystems is an established method in empirical software engineering (Zerouali et al. 2022). We report chance-corrected agreement (Cohen 1960; Fleiss 1971), interpreted on the scale of Landis and Koch (1977). Software-engineering studies that test whether LLMs can replace human annotators or evaluators find that this works for some tasks and not for others (Ahmed et al. 2025; Wang et al. 2025), so we report model raters next to the human rater, not instead of it. Empirical studies of vulnerabilities in LLM-integrated software exist; for example, Liu et al. (2024a) study code-execution flaws in LLM-integrated frameworks and apps. They classify vulnerability types, whereas RQ1 classifies the trigger point of a disclosure (model decision or client and service code) on a sampled window of advisories and reports agreement between raters. We searched for earlier disclosure taxonomies for AI agents

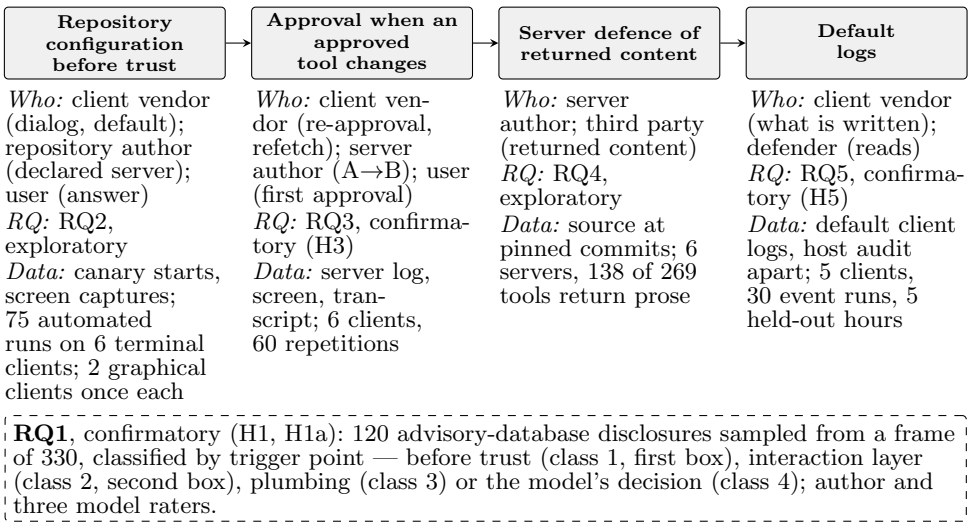


Fig. 1 Study overview: the four boundaries measured in a coding-agent session (left to right), who controls each, the research question that measures it and its data. RQ1 gives context from public disclosures.

that report inter-rater agreement. The search was not systematic, and we make no claim that none exist.

Telemetry and detection.

Most of the work above reports attack success rates, not whether a defender could see the attack. AgentSight observes coding agents at the system level with eBPF (Zheng et al. 2025). Chen et al. (2026) test OS-level controls and detectors against attacks on an agent's own state using Linux telemetry. They find that detectors flag a substantial part of legitimate activity. RQ5 complements these studies with a narrower question: whether the *default* client logs separate the RQ2 and RQ3 events from normal use, measured as a false-positive rate on a baseline; host auditing added by the lab is reported apart.

4 Study design

Preregistration.

Research questions, hypotheses, classification criteria (Appendix A), raters, analysis plan and the protocols of RQ3 and RQ5 were frozen on 11 October 2026 (Costanzo 2026c); Fig. 1 gives an overview. RQ2 had been measured, and an approximate reading of RQ4 made (later withdrawn), before that date; both are reported as exploratory, as is a ten-case pilot of RQ1. The RQ4 figures come from a per-tool census made after the freeze at pinned commits, under coding rules fixed during the census. The 60% threshold of H1 was set after the author's own classification of the pilot cases (published in a blog post on 29 September 2026; Costanzo 2026b), so H1 is a weak test; the stronger confirmatory evidence for RQ1 is the agreement between raters

(H1a) and the share on a new random sample that excludes the pilot. The frozen v2 revised a v1 draft of 10 October after the pilot had been run (preregistration §9): H1a, H3 and H5 were made testable, so H1a's threshold (the conventional 0.6) was fixed with the pilot's Fleiss' κ of 0.91 known, and a hypothesis on RQ4, stated after its measurement, was removed. Deviations from the preregistration (D1–D56) are logged with date and reason in the public artifact repository and listed in Online Resource 1, Sect. D, with which data existed when each was written: D1–D18 were written before any confirmatory data, D40 before any RQ1 confirmatory data and D41–D48 before the RQ1 model runs; D19–D39 while or after the RQ3 and RQ5 campaigns ran; D49–D56 after the RQ1 run had ended (D49 records a decision taken during it), D50–D56 after all confirmatory results; D50 also records that the analysis code was extended after the results to report robustness and run-to-run instability (no figure changed). D1–D47 were pushed publicly when committed; D48–D56 were committed locally and first published with v0.5.0 on 1 November 2026, so their dates rest on local commit times (D26 was first committed on 22 October, five days after it was written; D50 on 27 October although its header reads 28 October; D52's design was committed locally before its cells ran). Detail in Online Resource 1, Sect. E.1. Analyses added after all results are not preregistered: an audit of the RQ1 class-3 labels, boundary cases and flags; the RQ3 outcomes read as observable states; a per-tool reading of partial coverage in RQ4; the log coverage of the RQ5 held-out windows; and a reading of the RQ1 primary sources with an explicit "cannot tell" outcome, prevalence-robust agreement coefficients for RQ1, a register of decisions and the reproduction of the analyses without git history (D55, D56). They are labelled post hoc and exploratory where they are reported, change no confirmatory estimate or verdict, and their files are in the artifact (Costanzo 2026a). Table 1 lists the decisions that could have affected a confirmatory verdict, with when each was made and which data were visible; the full register, `preregistration/CHRONOLOGY.md` and `chronology.csv` in the artifact, has 86 decisions (42 prospective, 11 made during collection, 33 after results), and none of those made after results changed a confirmatory figure or verdict.

Ethics, disclosure and dual use.

All clients, canary servers and self-hosted models ran on machines controlled by the author; clients configured with hosted models sent their prompts to the vendors' services through the author's own accounts and keys. Every canary is harmless: it records that it started and answers a minimal handshake with one no-op tool. The study describes default behaviours at the level of product, version and observed behaviour, without attack procedures; new security-relevant observations were reported to the vendors' security channels before they were discussed in the blog series or in this paper, and behaviours already documented by vendors are listed as such (Appendix C). No human participants were involved. The lab results were first published by the author as a numbered blog series, starting with the pilot post of 29 September 2026 (Costanzo 2026b,d).

Table 1 Decisions that could affect a confirmatory verdict (short view of the chronology in the artifact; D-numbers as in Online Resource 1, Sect. D). "Prospective": before the data it governs.

RQ	Decision	When	Data visible	Effect on verdict
RQ3	H3 clause 2 read as "notifies or asks" (D12)	Prospective	None	Changed: clause 2 fails (holds under frozen wording)
RQ1	Contested cases counted against/for H1 (D41.6)	Prospective	None	Changed: H1 not robust (32.6%)
RQ1	Population: random sample of 120 from frozen frame (D44)	Prospective	Frame only	Changed: estimate for the frame
RQ1	H1 threshold 60%	Prospective	Pilot (7/10)	Could change; observed 90.1%/92.6%
RQ1	H1a threshold $\kappa \geq 0.6$	Prospective	Pilot (κ 0.91)	Could change; observed 0.458
RQ1	No-majority against H1; R2 digest differs (D7, D43)	Prospective	None	Without R2: point estimate only (73.4%)
RQ1	R0 input: AI summaries, Italian, guided questions (D42, D46, D47)	Prospective	None	Could change H1, H1a
RQ1	Pilot worked examples before R0 (D50.1)	Before labels; logged after	None at decision	Could change H1a
RQ3	One repetition excluded and re-run (D19)	During collection	Runs, no outcome read	Could change (1/60)
RQ5	Threshold $< 5\%$, per-event aggregation, freeze (D27)	Prospective	None	Could change H5
RQ5	Cursor CLI deferred: five clients (D29)	Prospective	Set-up sessions	Could change H5
RQ5	Set-up sessions as rule material; verdict code (D31, D36)	During collection	Event runs, no holdout	Could change H5

Attacker model.

No attack outcome is measured; each lab question fixes who controls what and lets a harmless canary stand for the step that matters. *RQ2:* the attacker controls the content of a repository, including its project configuration files; the user opens it and answers the client's first question; a canary start stands for execution of a command chosen by the repository's author. *RQ3:* the attacker controls an MCP server the user has already approved under definition A and later serves B; the user only continues the session or reopens the client; the no-op description change stands for a change of the text the model reads about the tool. That a stdio server already runs code on the host does not make RQ3 moot: an approval is given to what the user saw, a server benign at approval can serve another definition after an update (the after-restart column of Table 7), and a changed definition acts through the model, not through a new process, so process auditing does not see it (host audit 0/10 for each kind of change, RQ5). The HTTP transport, where the server does not run on the user's host, was planned but not run. *RQ4:* the attacker controls third-party content (an issue body, a log, a document) that an official server returns to the model; the census asks only whether that output is routed through the server's own defence, not whether the defence would stop the injected text. *RQ5:* the attacker is the RQ2 or RQ3 event; the defender has the client's default logs (and, reported apart, host auditing configured by the lab); we assume the logs are not altered, which we did not test. *RQ1* has no lab attacker: the triggering input of each disclosure and who supplies it are taken from its summary, and they decide the class.

Lab.

A Debian 13 virtual machine restored from snapshots, with process and trust-store auditing and egress logging; exact client versions; open-weight models self-hosted with Ollama on a rented cloud GPU (one NVIDIA L40S), with recorded tags and digests.

Use of AI assistants.

AI assistants were used throughout, and every use is part of the method; none is copy editing only. *Claude* (Anthropic; Claude Opus 5.5 where the log records the model) wrote the code of the lab and its analyses (it is a co-author of every commit of the artifact repository): the harness and runners, the canary servers, the rule-based RQ3 coder and the coding packets, the RQ5 normaliser, rule applier and scoring tools, and the reproduction script. It ran the lab, including the two graphical clients, whose keystrokes it sent (the author signed in to their accounts and, for RQ3, watched the console). It drafted the text of this paper, of the deviations log and of the vendor reports, and carried out the post hoc analyses (Section 4, Preregistration); for the RQ1 source audit it made the first reading and the frame check, and Codex made the blind second reading. For RQ1 it screened and grouped the frame under rules written beforehand, wrote the case summaries from primary sources and translated them and the criteria into Italian for the author's labelling. For RQ4 it made the first and the third reading of the census; the third reading proposed a decision for each of the 91 disputed tools. For RQ5 it wrote the primary detection rules, called the author's rules below; a fresh session of the same model, with no history, was the second rule author, blind to those rules and to the held-out data. *Codex* (OpenAI; for the RQ4 coding, Codex CLI with model `gpt-6.1-sol`, reasoning effort medium) reviewed plans, code and analyses, checked every RQ1 summary against its sources and every Italian translation against its English original, coded RQ3 sessions as an additional coder, made the second reading of the RQ4 census, and re-computed the RQ1 analysis independently from the raw files. Three local open-weight models are the RQ1 raters; a local model is also a preregistered RQ3 coder. Before submission the manuscript was checked by LLM assistants (Claude and Codex in fresh sessions); these are pre-submission checks, not peer review, and their limits are stated in Section 11. *The author* directed the study: he accepted the design proposals and approved the preregistration, with its research questions, hypotheses and protocols, before its freeze; chose the RQ1 sample design of D44; approved the publication of the artifact repository and its deviations log; labelled the 120 RQ1 cases as R0 before any model ran; examined the RQ3 evidence against the rule-based outcomes and spot-checked ten seeded rows of the RQ4 census; decided the 91 disputed RQ4 tools by consulting the cited source (taking the third reading's proposal in 90); approved the publication of the candidate RQ5 rules before the baseline sessions; and sent the vendor reports. No language model is an author; the author takes responsibility for the whole work. Table 2 summarises the roles. Because the assistants are made by vendors of evaluated clients (Anthropic: Claude Code; OpenAI: Codex CLI; Google: Gemini CLI and the Gemma rater; Alibaba: Qwen Code and the Qwen rater), decisions that could favour a vendor rest on primary sources: advisories, CVE records and the vendors' own documentation for RQ1 and RQ4, and the server logs and ground-truth captures for RQ3 and RQ5, where an assistant's vendor

Table 2 Use of AI assistants by task (models as recorded in the project logs; "—": no human role reported for the task).

Task	AI tool (model)	Human role
Lab code, runners, analyses, reproduction	Claude (Opus 5.5 where logged)	author accepted the design proposals
Running the lab	Claude	author signed in to graphical clients, watched RQ3 console
RQ1 frame, summaries, translations	Claude, under rules written beforehand; checked by Codex	— (no human screened the candidates)
RQ1 labels	three open-weight models (R1–R3)	author labelled all 120 (R0) before any model ran
RQ1 re-computation	Codex	—
RQ1 source audit (post hoc)	Claude, then Codex blind	— (no second human)
RQ3 coding	rule-based coder (Claude-written), local model, Codex	author checked evidence against outcomes
RQ4 census	Claude (1st, 3rd reading), Codex (2nd)	author decided 91 disputes by consulting the cited source
RQ5 rules	Claude (primary), fresh Claude session (second)	author approved publishing the candidate rules
Paper, deviations log, vendor reports	Claude drafted; Claude and Codex checked	author sent the vendor reports

also makes a measured client (Codex coded Codex CLI sessions; Claude wrote rules scored on Claude Code logs). Before submission Claude listed 36 key results with their sources and drew 20 of them with a script using a declared seed; the author compared each drawn figure with the source shown next to it (the numbered file lines for 12 figures; for the 8 counts, a short command run on the source files, shown with its code); all 20 matched.

Raters and agreement.

R0 is the author. R1 Qwen3.8-27B (q8_0), R2 Gemma 4 26B-A4B (q8_0) and R3 Nemotron 3.5 Lightning 30B-A3B (q4_K_M) run in Ollama (sha256 digests `8f5fb6b71ea0...`, `7473a33e49f1...`, `e7a64ff15fb1...`, full values in the run manifest) with temperature 0, a context of 16,384 tokens and three runs (seeds 1–3); each model's class is the mode of its valid answers (invalid answers are dropped; for three Gemma cases this leaves a single answer, and every case with an invalid answer is contested). The preregistered build of R2 (`6bfaf9a8cb37`) was no longer available under its tag; the current build is used and every R2-dependent result is also reported without R2 (D43). Cohen's and Fleiss' κ (Cohen 1960; Fleiss 1971) are interpreted on the original bands of Landis and Koch (1977); the preregistration grouped all values below 0.4 as 'poor', so on that scale Nemotron's 0.328 and the RQ3 coder agreement of 0.115 are poor. No test depends on these labels.

5 RQ1: where failures trigger

Classes.

Each disclosure is placed in one of four classes by trigger point: (1) *before trust*, code that runs at startup or before the user's trust decision; (2) *interaction layer*, an approval or confirmation step that misrepresents or skips what happens; (3) *plumbing*, authentication, authorisation or origin flaws in the services around the agent (MCP servers, SDKs, connectors, bridges); (4) *authority*, the model is manipulated and the impact is set by the privileges it holds. A case is class 4 only if the attack works because the model follows instructions it should not have trusted; otherwise it goes to the earliest step that fails without any model decision. The frozen text is in Appendix A.

Pilot (exploratory).

The blog post (Costanzo 2026b) and the preregistration report a pilot share of 7/10 in classes 1–3, with C01 (CVE-2026-55607, GHSA-7835-87q9-rgvv, a worktree sandbox escape in Claude Code) in class 1 (Anthropic 2026b). The advisory states that reliable exploitation required the user to clone a malicious repository containing prompt-injection content and run the agent on it; under the written decision rule this is class 4, and the author's pilot share is 6/10 (6/9 if the case is dated by its advisory, which falls before the window). The class-1 sentence on the agent's own execution machinery (sandbox boundaries, path resolution) was added after the pilot because of this case; the frozen preregistration is not modified (D14). The pilot cases are excluded from the confirmatory frame.

Frame and sample.

Deviation D44 replaced the preregistered population (the author's timeline of disclosures) with the keyword query of deviation D1 (written before any confirmatory data) over the GitHub Advisory Database and NVD: only one of the 330 frame cases comes from the timeline, so the result is an estimate for advisory-database disclosures. The screening rules were written after the query had run and before any candidate was read, and the final text of D44 was revised after the frame was built. The dated query (25 October 2026) returned 695 candidate records. Screening against the inclusion criteria (a primary source; a tool-using agent, an MCP server or connector, or an agent SDK; disclosure in the window), a second reading of 43 uncertain records, the grouping of records into disclosures (one disclosure is one case, also across products), the exclusion of every pilot component and alias, dating each case by its first public advisory or researcher write-up (142 cases fell before the window) and an eligibility check of every case against its primary source left 330 cases: 100 agent clients, 210 MCP servers or connectors, 20 SDKs (counts per step in Online Resource 1, Sect. E.2). A stratified random sample of 120 cases was drawn: proportional allocation by product type (37, 76, 7), with a seed fixed by a public randomness beacon (a drand round) that did not exist when the frame was published (Online Resource 1, Sect. E.2). The draw can be recomputed offline from the artifact, with or without its git history (Declarations). Nothing is replaced after the draw. The frame, its inputs and the rules are public (Costanzo 2026a).

Summaries and labels.

Each sampled case is described by a summary of 50–150 words written from its primary sources by Claude, stating every precondition and who supplies the input that triggers the flaw, without product names, identifiers or the words of the taxonomy; Codex checked every summary against copies of its sources and corrected 51 (D42). Cases are flagged before labelling when the check left a precondition, the triggering input or (in three cases) the disclosure date or a source's identity in question: 50 of the 51 corrected summaries and 24 unchanged ones whose sources stayed uncertain (74 of 120). The models read the English summaries. The author labelled every case on a sheet that showed no proposed class or model output, in an order fixed by the same seed, and committed the labels before any model ran; he opened no primary source while labelling. Three aids were added before labelling and are reported as deviations: an Italian translation of summaries and criteria by Claude, checked by Codex, with the English text one key away; questions that walk through the decision rule in its own words, leaving the class to the author's answers — in the event the author chose 117 of 120 classes directly and 3 from the questions, none contradicting the recorded answers; and worked examples taken only from the pilot cases and their already published labels (the model raters had no examples). One sampled case was found after the draw to predate the window; it stays in the primary analysis and is dropped in a sensitivity analysis.

Runs.

The model raters ran on 27 October 2026 (16:19–23:10 CET) on a temporary copy of the lab GPU machine (same disk image) in another availability zone of the same region, with the model digests checked against the frozen roster and recorded in the run manifest; no transient error occurred, and the run's time limit was extended during the run with the author's approval, which changed only how long the machine stayed on (Sect. E.2).

Analysis.

The share in classes 1–3 is estimated for R0 and for the model majority by the stratified estimator, with a conservative exact 95% interval (hypergeometric bounds per stratum, Bonferroni across strata; a case without a label widens it both ways). H1 holds when the point estimate is at least 60% for both. Each preregistered variant is computed alone against the primary analysis. A case is contested when R0 and the model majority differ, when there is no majority, when a model's mode is tied or includes an invalid answer, or when its summary was flagged; if counting the contested cases against or in favour of H1 changes the verdict, H1 is reported as not robust (D41). H1a: Fleiss' $\kappa \geq 0.6$ on cases with complete ratings. Codex re-computed every confirmatory number of this section from the raw files with its own calculations, without using the analysis code, and found no numerical difference; it found two reporting omissions (the D41 robustness statement and run-to-run instability), both since added. Its final report is in the artifact (`preregistration/rq1/codex-recomputation-20261027.md`).

Results.

Table 3 gives the primary estimates. The author placed 90.1% of the frame in classes 1–3 (interval 77.9–95.8%) and the model majority 92.6% (81.2–98.2%), so H1 is supported, with both intervals above 60%. H1 is *not robust*, however: 78 of the 120 cases are contested, and counting them all against H1 lowers both shares to 32.6%; counting them in favour raises both to 97.5%. Most contested cases (74) are summaries flagged before any label was given (Summaries and labels; the flag reasons are published); with all of them counted as class 4 the share cannot exceed 38.4%, so this variant was bound to fail H1 once the flags were set. Every other variant keeps the verdict (Table 4; two preregistered variants did not apply, since no summary was corrected after the model runs and no grouping error was found in the frame); without R2 the model majority falls to 73.4% and H1 holds on the point estimate only. This drop comes from the aggregation rule, not from the raters: with two models a majority needs both to give the same class, and every case without one counts against H1; Qwen and Nemotron differ on 31 cases, 26 of them within classes 1–3, while each alone places at least 95% of the sample in classes 1–3. Separately from the preregistered variants, a post hoc check of the saved sources (exploratory; Section 11) found 28 of the 46 cases with residual ambiguity between classes 1–3 and 4 whose sources do not establish whether a model step is needed; counted as class 4, they lower the share to 69.3% (R0, 55.2–79.7%) and 71.0% (model majority, 56.7–81.5%), above 60% on the point estimate but with both lower bounds below it. This check does not change the preregistered verdict above; a fuller reading of the sources follows. H1a is not supported: Fleiss' κ over the four raters is 0.458 (0.408 without R2; exploratory bootstrap 95% interval 0.35–0.55), moderate agreement on the scale of Landis and Koch (1977). Against the author, Qwen agreed best ($\kappa = 0.654$, substantial), then Gemma (0.513, moderate) and Nemotron (0.328, fair). The raters differ mostly in how they divide classes 1–3 and in how often they use class 4 (Table 3): Nemotron chose class 4 for one case, the author for twelve. Three cases have no model majority and count against H1. Qwen and Nemotron gave the same valid class in all three runs of every case; Gemma differed between runs on four cases and gave an invalid answer on five (eight answers, 2.2%, below the 10% limit of the plan); all eight are empty answers whose generation reached the context limit. By product type, the share is 83.8% for agent clients (R0 and majority; intervals 72–92% and 72–97%) and 92.1% (R0, 86–96%) and 96.1% (majority, 90–99%) for MCP servers and connectors; the seven SDK cases are not interpreted. H1 does not depend on any single stratum: the frame without MCP servers and connectors gives 86.5% for both raters, without agent clients 92.8% and 96.4%.

Source audit (post hoc, exploratory, D56).

Made after all results and not preregistered, this audit asks whether the primary sources themselves document the route, with an explicit "cannot tell". Each of the 120 sampled cases was decided from its saved primary sources only, blind to every label, as A (the source documents a sufficient chain without model manipulation), B (the documented chain needs the model to follow untrusted instructions) or C (the source does not allow a decision). Claude read every case first (A 77, B 12, C 31); Codex, a model of another provider, then read them blind. The two readings agree exactly on

101 of 120 cases and oppose A to B on only one; in 17 of the other 18, Codex decided A where Claude could not tell. Jointly, 76 cases are A (all in R0's classes 1–3), 11 are B (9 in R0's class 4, 2 in classes 1–3) and 33 are undetermined or disputed (30 in classes 1–3, 3 in class 4). With the frame weights and the unmodified estimator, the share of A is 86.9% (70.6–94.8%) among the 87 cases both readings decided — descriptive only, since the decided cases are not a random draw — and 63.5% (49.1–74.5%) with the undetermined cases counted against. The sources thus document a route outside the model for about two thirds of the frame, and for 33 of the 120 cases they cannot settle it; the preregistered 90.1% rests partly on cases whose sources do not document the route. A frame check made alongside raised a conditional doubt about the counting unit of one case (a cross-product bundle, which the frame rules permit) and found four possibly ineligible cases; no label and no frame row was changed. Both readers are AI systems, so this is an assisted check of the sources, not an independent validation; the preregistered verdict is unchanged.

Where the raters disagree (exploratory, D51).

An analysis written after the results, which changes no confirmatory estimate or verdict above, locates the disagreements. Of the 13 cases where the author and the model majority (built from the three models) give different classes, eight are between classes 1 and 3 and one between 2 and 3; four concern the question H1 asks, and in all four the author chose class 4 and the majority a class in 1–3. Agreement on that binary question is not much better than on four classes, however: Fleiss' κ over the four raters is 0.48 (bootstrap 95% interval 0.27–0.64), against 0.458 (0.35–0.55) on four classes; the binary point estimate is also below 0.6, but its interval is too wide to rule substantial agreement in or out, so this exploratory measure neither rescues nor further refutes H1a. Because about nine labels in ten fall in classes 1–3, raw agreement is high (on the binary question the author and the majority agree on 113 of the 117 cases that have a majority, against 104 of 117 on four classes) while κ stays low and uncertain. Prevalence-robust coefficients, computed post hoc and reported alongside κ, not in its place (Feinstein and Cicchetti 1990; Cicchetti and Feinstein 1990), are higher: Gwet's AC1 (Gwet 2008) over the four raters is 0.770 (bootstrap 95% interval 0.71–0.83) on four classes and 0.931 (0.89–0.97) on the binary question; for R0 against the model majority (117 cases), AC1 is 0.876 on four classes and 0.961 on the binary question, and the binary PABAK (Byrt et al. 1993) is 0.932 (Table 5). H1a, preregistered as Fleiss' $\kappa \geq 0.6$, is still not met. Nemotron, which chose class 4 once, has a binary κ of 0.14 against the author. Of the 78 contested cases, 74 are contested because their summary was flagged; the other four are three disagreements between the author and the majority and one invalid model answer. Disagreement is somewhat more frequent among the flagged summaries (10 of the 71 flagged cases that have a majority, against 3 of 46; two-sided Fisher exact $p = 0.24$, compatible with chance at this size), and the three cases without a majority are all flagged. The script, its tests and a labelling sheet for a second, independent human rater are in the artifact; no second human has labelled the cases yet.

Table 3 RQ1 primary analysis: classes per rater over the 120 sampled cases (models: mode of three runs; 1 before trust, 2 interaction layer, 3 plumbing, 4 authority), share of the frame in classes 1–3 (stratified estimate, frame of 330), conservative exact 95% interval, and the unweighted sample share.

Rater	1	2	3	4	Share (stratified)	95% interval	Unweighted
Author (R0)	6	2	100	12	90.1%	77.9–95.8%	90.0%
Model majority[a]	14	4	93	6	92.6%	81.2–98.2%	92.5%
Qwen (R1)	11	2	101	6	95.0%	84.2–98.2%	—
Gemma (R2)	14	5	90	11	90.9%	79.4–95.8%	—
Nemotron (R3)	31	5	83	1	99.2%	90.3–99.7%	—

[a] 3 cases without a majority.

Table 4 RQ1 variants, each computed alone against the primary analysis. "Supported": point estimates at least 60% for both; the interval note says whether both lower bounds clear 60%. Contested cases: 78 (74 flagged summaries, three further disagreements between the author and the majority, and one invalid model answer).

Variant	R0	Majority	Verdict
Primary	90.1%	92.6%	supported, interval above 60%
Mixed bundles counted as class 4 (D45)	88.4%	90.9%	supported, interval above 60%
Contested cases against H1 (D41)	32.6%	32.6%	not supported
Contested cases in favour of H1 (D41)	97.5%	97.5%	supported, interval above 60%
Model majority without R2 (D43)	90.1%	73.4%	supported, point estimate only
Without cases kept in doubt after screening (D44)	90.0%	92.5%	supported, interval above 60%
Without the late-ineligible case (D44)	90.0%	92.5%	supported, interval above 60%
Labelled cases only (D44)	90.1%	94.8%	supported, interval above 60%
Equal weights for the three product types (D44)	92.0%	93.3%	supported, interval above 60%

Interpretation.

Most sampled disclosures fall in class 3 (plumbing): conventional code flaws in the servers and services around the agent that do not need the model to follow untrusted instructions. In a post hoc audit (exploratory), 42 of the author's 100 class-3 labels fit the class-3 clause literally (authorisation 21, authentication 15, origin 6). The other 58 (unconfined file paths 23, unrestricted request destinations 12, attacker text reaching a command 9, other 14) are in class 3 by reading the frozen decision rule as exhaustive: it offers no class beyond 1–4 for a case that needs no model step and is not in class 1

Table 5 RQ1, R0 against the model majority (post hoc, D56): rows R0's class, columns the majority's class. 1 before trust, 2 interaction layer, 3 plumbing, 4 authority. On the 117 cases with a majority, raw agreement is 104/117 on four classes and 113/117 on the binary question (classes 1–3 against 4); Gwet's AC1 is 0.876 and 0.961, PABAK 0.852 and 0.932.

R0 \ majority	1	2	3	4	No majority	Total
1	6	0	0	0	0	6
2	0	2	0	0	0	2
3	8	1	90	0	1	100
4	0	1	3	6	2	12

or 2, while the wording of class 3 (authentication, authorisation, origin) does not cover them. We keep these labels and flag them; the replication package gives, for each, the violated property and the source evidence, written by an AI coder after all results and checked blind by a model of another provider; the two agree on all fields for 47 of the 58 (the disagreements, recorded and not resolved by relabelling, are summarised in Online Resource 1, Sect. E.3). Classes 1 and 3 are on the same side of the H1 split, so H1 does not depend on this, but the class distribution in Table 3 does; any wider reading of class 3 is exploratory. The frame is dominated by MCP servers and connectors (210 of 330), and the share among agent clients alone is lower (83.8%). The headline is therefore narrower than "failures happen outside the model": among advisory-database disclosures of one quarter for this ecosystem, most describe conventional flaws in the code around the model. The result is not robust to the contested cases, almost all of them summaries flagged before labelling (Results). Read by content (post hoc), 39 flags record a correction or a doubt that cannot change the class under the frozen rule, 32 a doubt that can (30 on the class 3/4 boundary), and 3 only dates or source identity. The flags record how sure we are of the inputs, not observed disagreement between classes 3 and 4: R0 and the model majority differ on the binary question in only 3 of the 71 flagged cases that have a majority (1 of 46 unflagged). For the 28 cases whose sources do not say whether a model step is needed (Results), a source that documents no model step does not show that none is needed, so for these cases the primary share may overstate classes 1–3. Agreement is only moderate both on the four classes and on the binary question, with most disagreements, by count, between classes 1 and 3.

6 RQ2: the trust boundary of repository-declared MCP servers

A test repository declares one harmless canary server in the six project configuration files; a server counts as started when the canary writes its start record. A manual pass covered all clients but Cursor IDE on 28 September. On 9 October 2026 a runner with fixed keystrokes drove the six terminal clients in 75 valid automated runs; a cell is one client in one condition (untrusted folder, after the trust decision, reopening a trusted

Table 6 RQ2: first question, pre-selected answer and canary starts (starts / repetitions). Versions: September / 9 October 2026 releases (Claude Code 2.1.283 / 2.1.295, Codex CLI 0.157.1 / 0.162.0, Gemini CLI 0.61.0 / 0.63.0, Qwen Code 0.24.6 / 0.25.0, Cursor CLI 2026.09.26, self-updated to 2026.10.01 during the first automated run, Goose 1.52.0; VS Code 1.139.1 with Copilot; Cursor IDE 3.22.7). Except for Gemini CLI, counts are for the September-version runs (for Cursor CLI, the runs of the first set); the single 9 October repetition of each cell gave the same outcome and is not added in. "Before": starts before the user answers, in the untrusted-folder cell (no other cell had a start before the answer).

Client	First question	Pre-selected	Before	After trusting
Claude Code	trust folder	No, exit	0/2	second, server-specific question; default declines (0/2; 2/2 if approved)
Gemini CLI	trust folder; names server	Trust folder	0/2, 0/4	in session 4/5 (0.61.0), 1/4 (0.63.0); on reopening 2/2 (0.61.0), 4/4 (0.63.0)
Codex CLI	trust folder; warns settings run code	Trust and continue	0/3	starts it, 3/3
Cursor CLI	trust workspace; says MCP enabled	Trust this workspace	0/2	starts it, 2/2
Qwen Code	per server, full command shown	Approve this server	0/2	starts after approval (2/2); asks again after a configuration change
VS Code + Copilot	on opening the chat	Trust Folder & Continue	0/1	starts it without a question when the chat needs a tool (1/1)
Cursor IDE	none	–	0/1	listed as disabled until switched on
Goose	none	–	–	does not read project servers (0/2)

folder, or a client-specific variant), and a run is one repetition of a cell. The first set (46 runs) used the late-September versions, except Cursor CLI, which updated itself from 2026.09.26 to 2026.10.01 during its first run; the set's Codex CLI cells (invalid in the full matrix, because a background-server screen of Codex replaced the trust question, and excluded) and its Gemini CLI cell after the trust decision were re-run after a snapshot revert. A second set of 29 runs used the releases of 9 October, with one repetition per cell except Gemini CLI (composition of both sets in Online Resource 1, Sect. E.4). The two graphical clients were operated interactively outside the runner (keystrokes sent by the AI assistant), once each in the configuration reported (VS Code with Copilot signed in, on 9 October; in the 28 September pass, without Copilot, VS Code started no project server on trust). Repetitions are not independent: the virtual machine was not restored between runs within a set, and runs that reopen a repository keep the decisions stored in earlier runs.

Results.

In no interactive cell did the canary start before the user answered the client's first question (Table 6; the exact wording of each question is in Online Resource 1, Sect. B). In five of the six clients that ask, the pre-selected answer is the permissive one; only Claude Code pre-selects "No, exit" and, after trust, asks a second, server-specific question whose default declines the server. In Codex CLI (3/3) and Cursor CLI (2/2), accepting the pre-selected answer started the declared server with no further question. Gemini CLI did so in the same session in 4 of 5 repetitions on 0.61.0 and 1 of 4 on 0.63.0, and at every reopening. Qwen Code asks per server, shows the full command, and asked again after a configuration change. With Copilot signed in, VS Code did not start the server on trust but started it without a question when its chat first needed the tool (one run); Cursor IDE listed the project server as disabled until switched on. In non-interactive mode, Claude Code's documentation states that the project server starts without any prompt (Anthropic 2026c), as TrustFall reported (Utevsky 2026b); on 2.1.295, two documented flags (`--bare`, `--setting-sources user`) prevented the start (0/3 each). *Note (weak evidence, not used as support elsewhere):* on 2.1.283 we observed the start in three runs on 28 September (3/3), recorded only in the author's results table; the canary logs of those runs are not in the released artifact. Re-checks of `claude -p` and `gemini -p` on the October releases were confounded by trust left from earlier runs and are excluded. Three further `claude -p` runs on 2.1.295, with the project's trust entry removed before each run, started the canary 3/3 but are also excluded: because of a script error the removal was not confirmed, and no per-run canary logs were kept. The other non-interactive clients stopped at an authentication or trusted-directory check.

Interpretation.

The trust dialog is the boundary, and the product's defaults decide what crossing it means: in four of the six clients that ask, one answer, pre-selected, covers both "trust this folder" and "run the commands it declares"; the information shown ranges from the full command to nothing; and behaviour can differ between releases weeks apart (Gemini CLI 0.61.0 and 0.63.0, few non-independent repetitions, cause unknown), so results are tied to versions.

7 RQ3: when an approved tool changes

A no-op server changes the description of an approved tool from A to B; the matrix crosses six terminal clients with two signal levels (no signal; `notifications/tools/list_changed`), a description change over stdio, hosted models (Gemini CLI, Qwen Code and Goose on Google's alias `gemini-flash-lite-latest`, the first two moved off their default model because of quota; the other three on their default model), and five repetitions, each from a clean running snapshot (12 cells, 60 repetitions, 15 October 2026; Claude Code 2.1.300, Codex CLI 0.162.1, Cursor CLI 2026.10.14, Gemini CLI 0.64.0, Goose 1.54.0, Qwen Code 0.25.1). Claude Code and Goose speak revision 2026-07-28 of the protocol, the others an earlier revision. Outcomes are first assigned by a rule-based coder in the

Table 7 RQ3 primary matrix (modal outcome over five repetitions; all 60 valid) and exploratory rows. "Stale": no new tool list fetched after the change, so the client held only definition A; "silent accept": fetched B after the change and called the tool without showing anything (what reached the model was not observed); "notified": shows a notice without asking again. First-call approval: "no question" = nothing asked; "persistent" / "for the session" = the option we chose in the client's question (Online Resource 1, Sect. B).

Client	First-call approval (our choice)	No signal	Notification	After restart (expl.)
Claude Code	no question	stale 5/5	silent accept 5/5	silent accept 10/10
Codex CLI	persistent	stale 5/5	stale 5/5	silent accept 10/10
Cursor CLI	persistent	stale 5/5	stale 5/5	silent accept 10/10
Gemini CLI	for the session	stale 5/5	notified 5/5	asks again 10/10
Goose	no question	stale 5/5	stale 5/5 (not subscribed)	silent accept 10/10
Qwen Code	no question	stale 5/5	stale 5/5	silent accept 10/10
VS Code + Copilot (interactive)	asks; Always Allow chosen	stale (1 run)	silent accept (2 runs)	silent accept (1 run, after the no-signal run)
Cursor IDE (interactive)	no question (snapshot settings)	stale (1 run)	silent accept (1 run)	silent accept (2 runs)

harness, which applies frozen per-client patterns to the server log, the screen and the transcript of each repetition; "stale" (the client keeps using definition A) is coded alongside notify, re-approval, silent accept and refuse; "acting on" the notification means notifying or asking in at least one of five delivered repetitions. A tool counts as approved once it has run at its first call, after the client's question where the client asks one (the option we chose is given in Online Resource 1, Sect. B) or with no question under the client's default auto mode. The preregistered coders are the author and a local model; a coder of another family was added after their agreement was known. The author did not label each packet blind: after examining the evidence he declared agreement with the rule-based coder's outcome on every repetition, and his labels are recorded from that declaration (D21), so they equal the rule-based outcomes. Changes to the schema or name, and the HTTP transport, were planned as matrix cells but not run; a rename was run later only as an exploratory positive control (end of this section).

Results.

Without a signal no client fetched a new tool list in the session and every call was made with only A served (30/30); the text each client showed is reproduced in Online

Resource 1, Sect. B. The 60 retained repetitions include one re-run: in the first attempt of Qwen Code, no signal, repetition 4, the client asked for permission after the change, with a message stating that its own permission classifier was unavailable; under a rule written after the matrix ended and after this attempt was seen, before outcomes were coded, the attempt was excluded and re-run. Its relation to the change is unknown: the client's message names the classifier's unavailability, and the attempt's server log was not kept; the question is not counted as a re-approval. Counting it would give one observed question in 61 attempts, and H3's first clause would be unchanged (post hoc sensitivity analysis). With the notification, the signal was delivered to five clients (server log); Goose does not subscribe. Claude Code fetched the new list 4–7 ms after the notification and called the tool without showing anything (5/5); Gemini CLI fetched it and showed a one-line notice without asking again (5/5); Codex CLI, Cursor CLI and Qwen Code were sent the notification, fetched no new list and kept calling with A (Table 7). Read as observable states (post hoc, exploratory), B reached a client within the session in 10 of 60 repetitions (Claude Code and Gemini CLI with the notification); of the primary-matrix clients that asked at the first call (Codex CLI, Cursor CLI and Gemini CLI; in Cursor CLI the harness chose "Allowlist MCP Tool" in all 10 primary repetitions, as recorded), only Gemini CLI received B in the same session, so for Codex CLI and Cursor CLI the absence of a second question reflects that no new definition was fetched; after reopening both received B and called it without a question (10/10 each). Whether any changed definition reached the model was not observed in these cells. No client required re-approval, so the first clause of H3 holds. The second clause, operationalised before data collection as notifying or asking (D12), does not hold: Gemini CLI acts on the notification in 5/5. Under the wording frozen on 11 October ("by asking the user") it would hold; we report the pre-data operationalisation as primary. After a restart (exploratory) five clients fetched B and called the tool without a question and Gemini CLI asked again, which with session-only approvals cannot be told apart from an expired approval. Agreement between the preregistered coders was low (author vs local model, Cohen's $\kappa = 0.115$, slight; 14 of 60 identical): 45 of the 46 disagreements trace to one ambiguous line of the coding packet written by Claude, and all were resolved on the server log in favour of the author's label; the additional coder agreed with the author on 60 of 60 ($\kappa = 1.00$; Fleiss' κ over the three coders 0.066). Because the author's labels are his recorded agreement with the rule-based outcomes, these values measure agreement with those outcomes, not with an independent human coding; no blind human coding of RQ3 was done. The two graphical clients, operated interactively, showed the same pattern. In exploratory cells with a local model (one repetition each, four clients), no client sent the changed description to the model before it was reopened. For Claude Code this rests on its outgoing requests only: the local endpoint rejected every main request with a server error, so the model never answered; Codex CLI made no tool call. With that caveat, "silent accept" describes the protocol exchange, not necessarily what the model sees.

Positive control (exploratory, D52).

Foreseen before any confirmatory data but run only after the results (one cell per client, Cursor CLI excluded), the control renamed the tool instead, so that the changed

tool was one the user had never approved. Gemini CLI showed its notice and asked for approval of the new name in the same session; Codex CLI kept the old name until reopened and then asked. The harness and the coder thus captured and coded the questions of these two clients after a change (Codex CLI with its frozen pattern, Gemini CLI with a pattern widened to the new name); the patterns of the other clients were not exercised after a change, and a rename tests the approval of a new name, not a second approval of a changed definition. In their default modes, Claude Code, Qwen Code and Goose showed no question or client notice for the renamed tool. After reopening, Claude Code's model mentioned the substitution in its reply; in the same session, Goose's model replied "ok" although its call to the removed name had failed with an unknown-tool error.

Interpretation.

Where a client asked at the first call, its approval persisted across the description change under the tested conditions within the session, and for Codex CLI and Cursor CLI also after reopening (after reopening Gemini CLI asked again, which cannot be told apart from an expired session approval); these observations do not show what the client stores or to what the approval is internally tied. Where Table C1 names such a key (rules by name in Claude Code and Qwen Code, an approval per server and tool in Codex CLI), it is taken from the vendors' documentation (Anthropic 2026a; Qwen Team 2026; OpenAI 2026), not from these runs. Claude Code, Goose and Qwen Code asked nothing in their default modes in these cells, so their approval is the mode's automatic one and there is no user decision to persist. The protocol lets definitions change, and no client re-asks within the session (after a restart Gemini CLI asks again, which cannot be told apart from an expired session approval); whether the model ever sees the new text depends on caching inside the client.

8 RQ4: coverage of untrusted-content defences

On 9 October 2026 we shallow-cloned fifteen official server repositories, that is, servers published by the vendor of the service they connect to, from a purposive list of eleven widely used vendors, plus the MCP reference servers; this is not a registry sample. Each repository's source, excluding tests and documentation, was searched for seven keywords (spotlight, untrusted, prompt injection, delimit, encapsulate, invisible, sanitize) and the matches were read; a server entered the census only if a defence is applied to tool output. Nine repositories had no defence found by this screen (one of them, Azure MCP, was archived at its cloned commit of 21 January 2026, so its successor monorepo was searched at a later commit, as a whole rather than server by server) and one does not publish its server's code. The census covers six servers, from four repositories, that do have one; one more repository with a defence is not reported here. Each registered tool was classified, at a pinned commit, by whether it returns free prose (third-party, or written only by the vendor, such as documentation; the latter is also reported separately) and whether that prose passes fully, partly or not at all through the server's own defence (D17). An output-side defence is a mechanism that marks or neutralises third-party content as data the model should not follow: delimitation

Table 8 RQ4: tool-by-tool census of the official MCP servers with an untrusted-content defence (pinned commits). Tools = registered tools that return free prose (third-party, or written only by the vendor, such as documentation). Full / Partial / None = the tool's prose passes entirely, partly or not at all through the server's own defence. Partial means that some returned fields, branches or content types bypass the defence; it does not mean the defence is ineffective where it applies.

Server	Tools	Full	Partial	None	Full only	Full + partial
Azure DevOps MCP	34	33	1	0	97.1%	100.0%
GitHub MCP	60	11	41	8	18.3%	86.7%
Sentry MCP	33	1	16	16	3.0%	51.5%
AWS IaC MCP	9	8	0	1	88.9%	88.9%
AWS SQL Server MCP	1	0	1	0	0.0%	100.0%
AWS Oracle MCP	1	0	1	0	0.0%	100.0%
Pooled over tools	138	53	60	25	53/138	113/138

together with an instruction to treat the delimited text as untrusted data, character hygiene, or provenance marking; delimiters that present the content as instructions to follow do not count. This rule was fixed after both readings and before any coverage figure from the per-tool census was published; it removed one candidate server from those with a defence. Two readings were made by AI agents from two model families (Claude, then Codex) and compared; the author adjudicated disagreements (Results).

Results.

138 of the 269 registered tools return free prose (Table 8); in 8 of them that prose is written only by the vendor. Before adjudication the two readings agreed on the class of 87.7% of the tools (Cohen's $\kappa = 0.76$), on the protection of 66.9% ($\kappa = 0.52$) and on the vendor-only flag of 97.4% ($\kappa = 0.62$), all computed over the 269 registered tools of the six published servers. On the 91 of these tools where the readings disagreed, the author decided each of them by consulting the cited source, taking the third reading's proposal in 90. The eight- and seven-server comparisons promised by D23 are not reported, because they include readings of servers whose figures are not published and could not be checked from the artifact (D53, written after all results). Full coverage ranges from 0.0% (the two AWS database servers, one partly covered tool each) and 3.0% (Sentry) to 88.9% (AWS IaC) and 97.1% (Azure DevOps). Over servers the mean is 34.6% with full protection only and 87.8% counting partial protection (median 10.7% and 94.4%); pooled over tools, 53/138 and 113/138, leaving 25 tools with no protection. Without tools whose prose is written only by the vendor the means are 33.2% and 86.8% (pooled 49/130 and 107/130; AWS IaC: 5 tools, 4 full, 1 none). These 130 tools are the third-party-content tools of RQ4; the headline figures also count the 8 tools whose prose is written only by the vendor. AWS was informed before publication and assessed these gaps as having no significant security impact. An earlier, approximate reading gave other shares; it was withdrawn in favour of this census.

Interpretation.

Where vendors ship a defence, it is applied unevenly across tools of the same server; typically, a tool is partial because the defence applies to some fields, branches or content types of its output but not to others: for example, the GitHub server filters invisible characters from issue titles and bodies but returns pull-request diffs and file contents unchanged, apart from an optional metadata label, and the Azure DevOps server delimits text attachments but returns other attachment types as unwrapped binary data (per-tool detail in the artifact). In a post hoc reading, 24 of GitHub's 41 partial tools have only that opt-in label, which is off by default, so in the default configuration no defence applies to their output; the census counts them as partial because their output is routed through the defence in the code, whatever the default configuration; counted as none, they would lower the pooled full-plus-partial count from 113 to 89 of 138 (GitHub 28 of 60). Other possible reclassifications, which would lower the count to between 107 and 94 of 138 with the median over servers unchanged at 94.4%, are recorded in the artifact and not applied. The figures say whether output is routed through the defence, not whether the defence resists attack, and with six servers the averages are descriptive.

9 RQ5: what a defender can see

For five terminal clients (Claude Code, Codex CLI, Gemini CLI, Goose, Qwen Code), three event types were produced in the virtual machine: a repository-declared server start (RQ2), an announced tool change and a silent tool change (RQ3). Claude wrote the primary detection rules, called the author's rules as in the artifact, on exploratory runs and a development hour; the client-log rules reached their final content on 22 October, the host rule's logic was written on 23 October, and its learning set of known commands was computed from development and set-up sessions only on 24 October. On 24 October all rules were frozen, i.e. committed with SHA-256 hashes together with the second rule author's rules and the scoring tools, before any held-out client log or holdout session was opened (D27; earlier reads of event-run records are listed below). The rules were then scored on 30 confirmatory event runs (two per client and event) and on one held-out hour of scripted normal use per client, in one-minute windows. As preregistered, a rule has to fire on every event run with a false-positive rate below 5%; the two-hour baseline per client was split into a development hour (for writing rules) and a held-out hour (for scoring), and the rate is the share of the 60 held-out one-minute windows with an alert (pass: at most two). This share of one-minute windows within one hour is how the preregistered "below 5% per hour" was measured; the two are not the same quantity. An event type is distinguishable only if every measurable client passes; H5 holds if one is not. A second rule author (a fresh model session), blind to the author's rules and to the held-out data, wrote its own rules for the primary rule applier and received the primary normaliser, both beyond the package inputs foreseen; two files the normaliser needs were left out of the package, so it tested its rules with its own time parser, and its frozen rules were then scored with the frozen scoring tools on the same material, with the same results. Gemini CLI, Qwen Code and Goose used `gemini-flash-lite-latest`, on the free tier for the

Table 9 RQ5: detection on default client logs with the frozen rules (detected runs / event runs), and host audit. "No event": the client did not start the server (ground truth). "No rule": the inspected default-log sources of the client (lab inventory) hold no record that tells the event apart (no stored tool definition, or nothing written at a server start), so no rule was written from them and no rate applies. Frozen scoring: no rule alerted in any of the 60 held-out windows. Post hoc coverage analysis (D55, not preregistered), counting only the windows that hold a record of the rule's source: unmarked detected cells, 0 alerts in 60 covered windows (Wilson 95% upper bound 6.0%); [a]0 in 46 (upper bound 7.7%, above 5%); [b]only 1 window covered, so the false-positive requirement is not adequately assessable (fewer than 30 covered windows).

Source	Server start	Announced change	Silent change
Claude Code	no event	2/2[a]	2/2
Codex CLI	2/2	2/2	no rule (0/2)
Gemini CLI	no rule (0/2)	2/2	no rule (0/2)
Goose	no event	2/2[b]	2/2[b]
Qwen Code	no rule (0/2)	2/2	2/2
Host audit (all clients)	6/6	0/10	0/10

event runs and on a paid key later; the served version is known only where the clients log it. Default client logs are the test of H5; the host audit, configured by the lab and therefore not default telemetry, is reported apart. Cursor CLI's RQ5 runs were moved to an exploratory addendum that follows in a later version of the preprint, so Cursor CLI does not enter the verdict.

Results.

Two pairs had no event (Claude Code and Goose did not start the repository-declared server), leaving 13 measurable pairs. On default client logs the frozen scoring counted 9 of 13 pairs as told apart from normal use: the event was detected in every run and no rule alerted in the held-out hour (Table 9); in the other 4 pairs the inspected sources gave no rule, so the event went undetected. An announced change was detected in all ten runs. A silent change was detected in Claude Code, Qwen Code and Goose by comparing a stored tool definition with the previous one; Codex CLI and Gemini CLI store no tool definition in the default-log sources inspected. For Qwen Code the stored definition exists only when the model calls the client's tool search before and after the change (it did so in 1 of 2 set-up sessions and in every confirmatory run of these two event types), so both Qwen Code results depend on model behaviour and not on the client's logs alone. A server start was detected in Codex CLI and left no usable record in Gemini CLI or Qwen Code. H5 holds: for two of the three event types, at least one client's default logs did not distinguish the event.

No frozen rule alerted in any of the 60 held-out windows of any client. In a post hoc coverage analysis (not preregistered; a pair is assessable for false positives only if at least 30 of its 60 windows hold a record of the rule's source, a threshold set after the coverage figures were known), 7 of the 9 pairs told apart were assessed on 46 to 60 covered windows; the two Goose pairs were covered in 1 window, because Goose keeps only its last ten model requests, so their false-positive side is not adequately assessable

and they are told apart on detection only. This does not change H5, which rests on pairs that failed on detection. With 60 windows, zero alerts have a Wilson upper bound of 6.0%, so the 5% threshold is met as an observed rate, not with confidence, and for Goose it is close to vacuous (1 of 60 windows covered). These bounds treat windows as independent; adjacent windows of one session may be correlated, so the bounds (also 0/46 and the host audit's 0/300) are optimistic. Vendor-documented retention of the default log sources (30 days by default for Claude Code transcripts and Gemini CLI chats, a fixed two weeks for Goose's CLI log, several locations undocumented) is listed in the lab inventory; every session started from a clean snapshot, so retention did not act within the lab. The blind second rule author reached the same outcome on all 15 pairs; it is a session of the same model that drafted the primary rules, and it received the primary rule applier and normaliser, including the per-client extractors that read tool definitions, and wrote its rules for them; both limit its independence, most of all on the tool-change pairs. Host audit, reported apart, recorded all six server starts (0 of 300 false-positive windows, 111 of them covered because the audit log rotates) and neither kind of tool change, which happens inside an already running process. With two runs per pair, detection rates are imprecise, and the false-positive rate measures scripted use without benign look-alikes.

No client log of a held-out run was opened before the 24 October freeze, but some event-run records were. On 22 October, before the author's rules reached their final content, the quota observer's records of two failed attempts (Qwen Code server start, Gemini CLI silent change) were read to decide whether they were quota stops. On 23 October, after the author's client-log rules were final but before the freeze, the ground truth of all event runs and every run's quota-observer record (which holds client screen captures) were read to check completeness and quota messages. Neither pair whose attempt was read before the client-log rules were final was detected (Table 9), and the author's client-log rules did not change after 22 October; the host rule and the event-time definition used for scoring were written on 23 October, after these reads; neither depends on whether the events happened.

10 Discussion

Decisions made by the product.

The main findings of this study come from the client and server measurements (RQ2–RQ5; Table 10): across them, several security-relevant choices were made by the client or the server: which answer is pre-selected, whether one decision also starts declared commands, whether an approval survives a change, whether a defence wraps a tool's output, and what the default logs keep. None of these is set by the model, and they differed between products, although what the logs end up holding can depend on what the model does (Qwen Code stores a tool definition only when the model calls its tool search, RQ5). Behaviour can also differ between releases weeks apart (RQ2, with few repetitions), and the protocol allows a definition to change silently (RQ3), so results are tied to versions. We did not vary the model systematically (the local-model cells of RQ3 are exploratory) or measure attack outcomes; the claim is about where decisions are made, not about how much of the risk they carry. RQ1 gives these measurements

Table 10 Results at a glance (figures from Sections 5–9).

RQ	Key result	Status	Main limit
RQ1	90.1% (author) and 92.6% (model majority) of the frame in classes 1–3; Fleiss' $\kappa = 0.458$; source audit (post hoc): 63.5% with undetermined counted against; 86.9% of the 87 decided (descriptive)	H1 supported, not robust; H1a not supported; audit post hoc	sources settle 87 of 120 cases; raters read AI-written summaries
RQ2	no interactive start before the answer; five of six clients that ask pre-select the permissive answer; in four, one answer covers folder and servers	exploratory	few non-independent repetitions; tied to versions
RQ3	no client asked again within the session after a description change (after a restart Gemini CLI asked again, 10/10); B reached a client in the session in 10 of 60 repetitions; Gemini CLI notified 5/5	H3 clause 1 holds, clause 2 does not (D12)	one change type over stdio; question capture validated for two clients
RQ4	full coverage 0.0–97.1% across six servers (median 10.7%); pooled 53 of 138 tools	exploratory	six servers chosen purposively; routing, not strength
RQ5	default logs told apart 9 of 13 pairs; silent change unusable in 2 of 5 clients, server start in 2 of 3	H5 holds	two runs per pair; Goose's false positives not assessable

context: under the preregistered analysis most advisory-database disclosures of the quarter fall outside the model's decision, but post hoc the primary sources document such a route for only 63.5% of the frame when the 33 undetermined cases are counted against (Table 10).

Relation to prior work.

RQ2 confirms observations of the TrustFall report (Utevsky 2026b): in Cursor CLI and Gemini CLI, trust was pre-selected and trusting the folder started the declared server (for Gemini CLI 0.63.0 in the same session in 1 of 4 runs, and at every reopening); for non-interactive Claude Code, the start without a prompt is stated by the vendor's documentation, and our own 2.1.283 observation is only a note (Section 6). It differs on interactive Claude Code: TrustFall reported trust pre-selected on v2.1.129, while 2.1.283 and 2.1.295 pre-selected "No, exit" and, after trust, asked a second, server-specific question whose default declines the server; we did not test 2.1.129 or TrustFall's configuration, so we do not attribute the difference. RQ2 also adds that on 2.1.295 two documented flags prevented the non-interactive start, that Qwen Code asks per server, and how the two graphical clients behave. RQ3 measures the client-side condition that rug-pull attacks rely on (Beurer-Kellner and Fischer 2025; Song et al. 2025; Yang et al. 2025). RQ5 agrees with system-level observation (Zheng et al. 2025; Chen et al. 2026) in that host process auditing saw every server start while some clients' default logs did not; it adds that neither source recorded every tool change.

For tool builders.

Separate the trust in a folder from the decision to run what it declares; show the command; do not pre-select the permissive answer; bind an approval to a definition, not a name, and ask again when it changes; route the whole output of every tool that

returns third-party text through the defence (in these servers, 81 of the 130 such tools are not fully covered); and log tool definitions with a digest.

For defenders.

Do not assume an agent's default logs record the events containment depends on: a silent tool change left nothing usable in two of five terminal clients, and a repository-started server left nothing usable in two of the three clients that started one, while host process auditing saw every server start and neither kind of tool change. Keep an inventory of each agent's log locations, verbosity and retention (RQ5 lists what the vendors document) and cover the rest with records the agent cannot alter, such as a gateway log of tool definitions with digests and process-start auditing. Ephemeral runtimes destroy the agent's logs with them; the lab copies them out with SHA-256 manifests before each reset. Verbose logging was not tested.

For evaluation practice.

Security evaluations of coding agents should name the client, its version and its configuration, not only the model; a benchmark result about a model says little about the product boundaries measured here. Table 11 lists what a result needs on record to be interpretable, including items this study recorded only after noticing their absence; each item comes from a difference observed in this study, and the table also reads it as a requirement for tools or evaluations, with a test that would verify it; none of these requirements was tested as a defence here. Online Resource 1, Sect. E.5 fills in the checklist for one client (Codex CLI) from this study's own records; it shows that the items can be filled from our data, not that the checklist has been validated externally.

Two cases from this study.

Whether B was loaded (RQ3, items 4–6). Codex CLI and Cursor CLI asked at the first call and did not ask again after the change, which read alone suggests that their approval survived a changed definition. In the session, however, neither fetched a new tool list with or without the notification (stale 5/5 in each cell), so there was no second definition to approve; only after reopening did both receive B and call it without a question (10/10 each), and only that observation bears on persistence. *Mode and trust state (RQ2, items 2–3).* Interactive Claude Code started no declared server before the answer (0/2) and its second question declines by default, while its documentation states that non-interactive mode starts project servers without a prompt; three non-interactive runs on 2.1.295 started the canary 3/3, but without a confirmed trust state they could be read either as a headless start or as leftover trust, and they are excluded (Section 6).

11 Threats to validity

A planned lab that was not run.

A lab on code that runs before the trust decision (Q1) was dropped because the AI assistant used for this work declined, under its usage policy, to prepare the test material; no Q1 result is reported. Also not done (D40): maximum-verbosity logging,

Table 11 Reporting checklist for security evaluations of AI coding agents, derived from this study: what to report, the observation that motivates it, the item read as a requirement for tools or evaluations with a test that would verify it, and what this study did not establish. No requirement is shown here to be an effective defence.

#	Report	Because (observation)	Requirement; *verification*	Limit
1	Client and exact version, separately from model and model version	Gemini CLI started the server in session 4/5 on 0.61.0, 1/4 on 0.63.0 (RQ2)	record client and model versions apart; disable self-update during runs; *same cell on two releases, each from a clean snapshot*	few non-independent repetitions; cause unknown
2	Interactive or non-interactive mode, with the exact flags	the vendor documents a start without prompt in non-interactive mode; on 2.1.295 two documented flags prevented it (0/3 each) (RQ2)	record mode and exact flags; *same repository in both modes, canary start as outcome*	few runs; the 2.1.283 note (3/3) has no released canary log
3	Trust state of the workspace and how it is reset between runs	in four of six clients that ask, one answer covers folder and servers; leftover trust confounded two re-checks (RQ2)	tools: separate folder trust from running declared commands; runs: record and reset trust state; *canary start counted after each answer; trust store reset and checked*	not tested whether separation changes what users approve
4	Stored approvals and the permission mode in force	within the session no client that asked at first call asked again; after reopening with B, Codex CLI and Cursor CLI did not ask (10/10 each) and Gemini CLI asked again (10/10); three clients asked nothing (RQ3)	tools: bind approval to a definition and ask again on change; runs: record permission mode; *change A to B, look for a question in the session and after reopening, with a positive control*	question capture validated only for Codex CLI and Gemini CLI
5	MCP servers with scope, transport and protocol revision, and a digest of every tool definition	a definition can change without the client asking again; B reached a client within the session in 10 of 60 repetitions (RQ3)	log a digest of each definition at every fetch; *compare logged digests with the server's log of served definitions*	only a description change over stdio; model input not observed
6	Whether change signals were requested and delivered	notification delivered to five clients, three fetched no new list; Goose not subscribed, and a notification never subscribed to is not evidence that it was ignored (RQ3)	report subscription and delivery from the server log; *server log of the notification and later* `tools/list`	one signal type
7	Which tools route third-party content through a defence	full coverage 0.0–97.1% across six servers, differing between tools of the same server (RQ4)	tools: route the whole output of every third-party-content tool through the defence; *per-tool census at a pinned commit, then an injection test per tool*	routing measured, not strength
8	Log locations, verbosity and retention, and how logs are kept before a reset	silent change unusable in 2 of 5 clients' logs, server start in 2 of 3; documented retention two weeks to 30 days; one client kept only its last ten model requests (RQ5)	log definitions with digests; inventory retention; copy logs out before reset; *event runs plus held-out baseline, false-positive rate per window*	two runs per pair; scripted use; verbose logging not tested
9	Account state: synced settings, plugins and extensions	the lab Claude Code account synced plugins and settings at login (Sect. 11)	record synced settings, plugins, extensions; *clean-account control*	not run
10	Repetitions and whether each starts from a clean snapshot	RQ2 repetitions not independent; RQ3 restored a clean snapshot for each	start every repetition from a clean snapshot; report n; *snapshot identifier per run in the record*	RQ3 five, RQ5 two repetitions per cell

a local-model baseline for RQ5, the secondary cells of the RQ3 matrix, a third RQ2 and a second RQ3 time point, a clean-account control and the RQ5 runs of Cursor CLI, which follow in a later version of the preprint. The per-client positive control for RQ3 was not run before the matrix; it was run afterwards (Section 7), on a later snapshot with the same client versions but different credentials, with the runner as amended after the excluded Qwen Code attempt, without Cursor CLI and with one cell per client.

Internal validity.

RQ2 repetitions are not independent (no snapshot restore between runs within a set); two non-interactive re-checks were confounded by leftover trust and a third (`claude -p` on 2.1.295, 3/3) cannot be verified; all three are excluded; the full matrix of the first RQ2 set used an earlier revision of the runner; the September non-interactive Claude Code observation (3/3) has no published canary log. The Claude Code account used in the lab synced plugins and settings at login; whether they affected the prompts measured is untested, since the clean-account control was not run. For RQ1, the summaries were written and checked by AI assistants, which is a weaker safeguard than the second human reader the preregistration foresaw "where available"; the author saw worked examples from the pilot that the model raters did not and labelled from translated summaries with guided questions; the raters read summaries, not sources. The RQ1 frame was also built by AI assistants: Claude agents screened the 695 candidate records, gave the 43 uncertain ones a second reading against their primary sources, grouped records into disclosures and checked eligibility against primary sources, and Codex reviewed the frame before it was pushed (one record added); no human screened the candidates. For RQ3, there was no blind human coding, so every agreement statistic involving the author measures agreement with the rule-based coder (Section 7); the additional coder was added after the packet-line issue was known, and one attempt was excluded and re-run under a rule written after it was seen. For RQ4, the first and the third reading were made by the same model family (Claude), the second by another (Codex); agreement on protection before adjudication was moderate ($\kappa = 0.52$), and the author, who does not program and used the proposals as a reading aid for the cited code (D38), took the third reading's proposal in 90 of the 91 disagreements, so the final labels are not independent of the AI readings and were not re-checked by a second human. They do not lean towards the first reading: on protection, the decision matches only the second reading in 57 of the 91 cases and only the first in 32. For RQ5, event-run ground truth and quota-observer records were read before the freeze, two of the latter before the client-log rules were final (Section 9); no held-out client log was opened before the freeze.

Construct validity.

A canary start stands for execution of a declared command; it says nothing about what a malicious server could do. "Silent accept" is measured at the protocol level. RQ4 measures routing through a defence, not its strength. For RQ1, a neutral summary is the rater's input, and the boundary between classes 3 and 4 depends on who supplies the triggering input; 74 summaries were flagged before labelling (Section 5; three of

them for the disclosure date or a source's identity, which do not bear on the class 3/4 boundary), and one further correction raised no flag. Each summary had to state that input, and all summaries were written by one assistant whose vendor also makes a client in the frame, while the author labelled without opening the sources; agreement therefore partly reflects a shared framing of the cases. The decision rule gives class 4 only when the attack needs the model to follow untrusted instructions; every other case, including conventional flaws that are not specific to agents, falls in classes 1–3. H1 therefore measures how rare model-only disclosures are in advisory databases, not how often agent-specific mechanisms fail. Of the 28 cases whose saved sources do not establish whether a model step is needed (Section 5), the author placed 25 in classes 1–3 and 3 in class 4, since the frozen rule sets no class for an unknown triggering input. In the source audit, two AI readers, one of the family that wrote the summaries, could jointly settle the route for only 87 of 120 cases (63.5% documented outside the model with the other 33 counted against), so how far the primary share rests on what the sources document, rather than on summaries, is uncertain, and this assisted check does not replace an independent human reading. For RQ3, "did not ask again" is informative only for clients that ask at the first call; outcomes are attributed to the client, but Gemini CLI and Qwen Code were moved off their default model and Goose ran on the same lab-chosen model. For RQ5, the frozen wording "default host telemetry" was measured on default client logs only; the one host source, process and trust-store auditing, was configured by the lab, so the host-audit row shows what added auditing sees, not what a default host records.

External validity.

One operating system (Debian 13), client versions of late September to mid-October 2026, one change type and one transport for RQ3, official servers only for RQ4, chosen purposively rather than sampled from a registry, with one repository that has a defence left out of the census, so RQ4 describes these servers and not official servers in general; five clients and scripted normal use for RQ5. Three clients (Gemini CLI, Qwen Code, Goose) shared one lab-chosen, unpinned Google model alias, the default for neither Gemini CLI nor Qwen Code, whose served version is recorded only where the clients log it; with another model, behaviour that depends on the model, such as when tools are called, may differ. The RQ1 frame covers advisory-database disclosures of one quarter, dominated by MCP servers and connectors (210 of 330). Its agent-client stratum mixes coding agents with general assistants and workflow platforms, so RQ1 is not an estimate for coding agents alone. The recall of the keyword query is unknown: disclosures in the databases that match none of the keywords are missing from the frame. Vulnerability datasets and NVD records have documented quality problems (Croft et al. 2023; Anwar et al. 2022), one reason every frame case was checked against its primary source. Advisory databases over-represent conventional code flaws that receive identifiers; findings about manipulation of the model often do not, which biases the frame towards class 3, as D41 anticipated.

Conclusion validity.

RQ1 is not robust to the contested cases and its 60% threshold was set after the pilot; under the preregistered aggregation a single client without the record satisfies H5, so H5 was a low-risk hypothesis; likewise, Claude Code, Qwen Code and Goose, in their default modes, asked no question in the retained cells, even at the first call (this does not show that they never ask: Qwen Code asked when its classifier was unavailable, Section 7), so for them re-approval was not observable and the first clause of H3 could fail only through the clients that ask at the first call, which makes it a low-risk hypothesis too; the positive control validated question capture only for Codex CLI and Gemini CLI; RQ3 and RQ5 rest on few repetitions per cell (five and two), RQ2 on two to five non-independent repetitions per cell in the first set (on the late-September versions, except Cursor CLI, which had already updated itself to 2026.10.01) and one on the 9 October releases (four for Gemini CLI 0.63.0), and the graphical clients on one. Model raters may share training data with the cases.

Researcher bias.

A single author, mitigated by the preregistration and its dated deviation log, model raters from three vendors, labels committed before any model ran, a re-computation of the RQ1 analysis independent of the analysis code (its report is in the artifact), and open artifacts. The author, who is also R0, had argued the thesis publicly with the pilot before rating, runs a company that offers offensive validation of agentic AI systems, and reports findings to vulnerability programmes that may offer rewards; all three favour finding decisions outside the model (see Declarations). The pre-submission checks were done by LLM assistants: the Claude sessions belong to the family that drafted the text, and Codex followed the project, coded RQ3 and made the second RQ4 reading. In RQ3 and RQ5 an assistant judged its own vendor's client; the RQ3 labels were settled on the server log and the RQ5 verdicts against the ground truth, both open.

12 Conclusion

For AI coding agents and their MCP servers, several security-relevant decisions are made outside the model: in trust dialogs and their defaults, in whether an approval survives a change, in which tools a defence covers and in what the default logs keep. In interactive sessions none of the eight clients tested (late-September to 9 October 2026 releases) started repository-declared servers before the user answered, but in four of the six that ask, the pre-selected answer trusted the folder and also let the declared server start; within a session, none of the clients that ask at the first call (15 October 2026 releases) asks again when an approved tool's description changes over stdio (two of them did not fetch the changed definition until restarted); in six official servers with an untrusted-content defence, the share of prose-returning tools fully covered ranged from 0.0% to 97.1%; and default logs miss some of these events. Tool builders control these choices, and security evaluations should report them (Table 11). As context, of the 330 advisory-database disclosures of one quarter about tool-using agents and their connectors that matched our keyword query, most about MCP servers and connectors,

an estimated nine in ten (90.1%, from a stratified sample of 120) fall in classes 1–3 under the preregistered analysis, mostly flaws in the code around the model (class 3), many not specific to agents; the result is not robust, raters agree only moderately on the class, and read from the primary sources (post hoc) the share documented outside the model is 63.5% when the 33 of 120 cases they leave unsettled are counted against. The measurements of client and server behaviour, not this share, carry the main findings.

Statements and Declarations

- **Funding:** No funding was received for this work.

- **Competing interests:** The author is co-founder and CEO of CoDe RTD, a company that offers research and offensive validation of agentic AI systems; none of the evaluated products is related to it. The author has reported findings to vulnerability programmes of measured vendors, which may offer rewards. Vendors of the AI assistants and model raters used in this work also make evaluated clients (Section 4).
- **Ethics approval and consent to participate:** Not applicable: no human participants or personal data were involved.
- **Consent for publication:** Not applicable.
- **Data availability:** Case summaries, labels, rater outputs, run manifests, analysis outputs, the RQ2–RQ5 evidence (with SHA-256 manifests for the RQ2 and RQ3 evidence, and run records for RQ4 and RQ5), and the preregistration with its deviations log are available in the artifact archive (Costanzo 2026a) and the preregistration record (Costanzo 2026c). The client captures cited in the paper (terminal text captures and the screenshots of the graphical clients) are part of the archived evidence, listed in its SHA-256 manifests. The two full GitHub Advisory Database dumps and the full advisory description texts of the RQ1 candidates are not in the release archives, and their SHA-256 hashes are published; the git history keeps the description texts in the frame commit; the reproduction does not need them. Raw records that hold lab-account data, such as the RQ5 client-log archives, are not redistributed (redacted extracts are available from the author on reasonable request; the saved copies of the RQ1 primary sources are listed with their URLs and SHA-256 hashes in `preregistration/rq1/source-audit-posthoc/source-copies.csv`); the redacted development material given to the second rule author is hashed in its package manifest, and the redacted RQ5 holdout and event archives used by the post hoc coverage analysis are hashed in `telemetry/coverage-posthoc/COVERAGE-POSTHOC.json`; the reproduction script names the inputs it cannot regenerate. One RQ2 cell (Claude Code 2.1.283, non-interactive, 28 September) is recorded only in the author's results table; its canary logs are not in the archive. Online Resource 1 (exact client prompts, the deviations log tables and protocol details moved from the paper) accompanies this article; the deviations log and the client captures it reproduces are in the artifact archive.
- **Materials availability:** Canary MCP servers and the test-repository generator are part of the artifact archive (Costanzo 2026a).

- **Code availability:** Harness, runners and analysis code: https://github.com/paolocostanzo/outside-the-prompt-lab, archived at https://doi.org/10.5281/zenodo.23300292 (concept DOI); the version used in this paper, v0.5.0, is https://doi.org/10.5281/zenodo.23861225. From the repository at tag v0.5.0, or from the release archive without git history, one offline command (`python3 reproduce.py`) re-runs the scripted analyses behind the reported figures (RQ1 estimates, variants and agreement; RQ2 matrix summary; RQ3 outcomes and coder agreement; RQ4 census and coverage; RQ5 scores and verdicts) from the published evidence, checks that each output equals the committed one (apart from the fields and rows the script names, such as absolute paths of the original runs and hashes of private inputs), and checks the RQ1 frame hash, seed and sample offline against the stored beacon value; it does not rebuild the frame or recount screening and summary checks. The RQ1 analyses read the frame, the sampler and the estimator as they are in the frame commit, whose SHA-256 values `frame-commit-pins.json` records; without git history the working-tree file is accepted only if it equals the pin, and otherwise the analysis stops (D56). `preregistration/REPRODUCTION-REPORT.md` labels every output as recomputed from public data, compared with a saved result, or not regenerable from public inputs. The script covers the scripted analyses that predate D55, exploratory ones included, and the Fisher test of D51; the post hoc analyses of D55 and D56 are run separately from their own folders (`*-posthoc`): the RQ3 states script works on the public evidence, while the RQ5 coverage script needs the private redacted archives and the saved source copies behind the RQ1 audits are private. This reproduces the analyses from the committed evidence (model answers, human labels and client captures are its inputs); it is not a replication of client behaviour, since no client, model or capture is re-run. The kinds of figures read directly from evidence tables, captures or documentation are named in the script, with pointers to the folders that hold them.
- **Author contribution:** Paolo Costanzo (ORCID 0009-0005-7205-5945) is the sole author. He directed the study, approved its design (research questions, hypotheses and protocols, in the preregistration) and chose the RQ1 sample design, labelled the 120 RQ1 cases as R0, examined the RQ3 evidence and spot-checked the RQ4 census against the recorded outcomes, adjudicated the 91 disputed RQ4 tools by consulting the cited source, approved the publication of the preregistration and of the artifact repository, made the vendor disclosures, and checked 20 randomly drawn reported figures against their sources. The code, the lab runs, the RQ1 screening and summaries, the RQ4 readings, the RQ5 rules, the post hoc analyses and the drafting of the text were done by AI assistants, as described in Section 4. The author takes responsibility for the manuscript.

References

Ahmed T, Devanbu P, Treude C, et al (2025) Can LLMs replace manual annotation of software engineering artifacts? In: 2025 IEEE/ACM 22nd International Conference on Mining Software Repositories (MSR), pp 526–538, https://doi.org/10.1109/MSR66628.2025.00086

Anthropic (2026a) Configure permissions (Claude Code documentation). https://code.claude.com/docs/en/permissions, accessed 17 October 2026

Anthropic (2026b) CVE-2026-55607 (GHSA-7835-87q9-rgvv): Claude Code sandbox escape via git worktree path confusion. https://github.com/anthropics/claude-code/security/advisories/GHSA-7835-87q9-rgvv, accessed 31 October 2026

Anthropic (2026c) Run Claude Code programmatically (Claude Code documentation). https://code.claude.com/docs/en/headless, accessed 31 October 2026

Anwar A, Abusnaina A, Chen S, et al (2022) Cleaning the NVD: Comprehensive quality assessment, improvements, and analyses. IEEE Transactions on Dependable and Secure Computing 19(6):4255–4269. https://doi.org/10.1109/TDSC.2021.3125270

Beurer-Kellner L, Fischer M (2025) MCP security notification: Tool poisoning attacks. https://invariantlabs.ai/blog/mcp-security-notification-tool-poisoning-attacks, Invariant Labs blog, 1 April 2025. Accessed 31 October 2026

Beurer-Kellner L, Buesser B, Creţu AM, et al (2025) Design patterns for securing LLM agents against prompt injections. https://doi.org/10.48550/arXiv.2506.08837, arXiv:2506.08837

Bhatt M, Narajala VS, Habler I (2025) ETDI: Mitigating tool squatting and rug pull attacks in Model Context Protocol (MCP) by using OAuth-enhanced tool definitions and policy-based access control. https://doi.org/10.48550/arXiv.2506.01333, arXiv:2506.01333

Byrt T, Bishop J, Carlin JB (1993) Bias, prevalence and kappa. Journal of Clinical Epidemiology 46(5):423–429. https://doi.org/10.1016/0895-4356(93)90018-V

Check Point Research (2025) CVE-2025-54136 – MCPoison Cursor IDE: Persistent code execution via MCP trust bypass. https://research.checkpoint.com/2025/cursor-vulnerability-mcpoison/, Check Point Research, 5 August 2025. Accessed 31 October 2026

Chen Y, Denis N, Di Pietro R, et al (2026) Self-state attacks on self-hosted AI agents: How far can OS defenses go? https://doi.org/10.48550/arXiv.2607.17986, arXiv:2607.17986

Cicchetti DV, Feinstein AR (1990) High agreement but low kappa: II. resolving the paradoxes. Journal of Clinical Epidemiology 43(6):551–558. https://doi.org/10.1016/0895-4356(90)90159-M

Cohen J (1960) A coefficient of agreement for nominal scales. Educational and Psychological Measurement 20(1):37–46. https://doi.org/10.1177/001316446002000104

Costanzo P (2026a) Outside the prompt — reproducible lab harness. https://doi.org/10.5281/zenodo.23861225, https://doi.org/10.5281/zenodo.23861225, version v0.5.0; concept DOI 10.5281/zenodo.23300292

Costanzo P (2026b) Outside the prompt: AI agent vulnerabilities of summer 2026. https://paolocostanzo.github.io/outside-the-prompt-ai-agent-vulnerabilities-summer-2026/, Blog post, first of the series "Outside the Prompt", 29 September 2026. Accessed 31 October 2026

Costanzo P (2026c) Outside the prompt: Measuring trust boundaries of AI coding agents outside the model — preregistration (v2). https://doi.org/10.5281/zenodo.23300369, https://doi.org/10.5281/zenodo.23300369

Costanzo P (2026d) Trusting a repo now means trusting its MCP servers. https://paolocostanzo.github.io/trusting-a-repo-means-trusting-its-mcp-servers/, Blog post, second of the series "Outside the Prompt", 13 October 2026. Accessed 31 October 2026

Croft R, Babar MA, Kholoosi MM (2023) Data quality for software vulnerability datasets. In: 2023 IEEE/ACM 45th International Conference on Software Engineering (ICSE). IEEE, pp 121–133, https://doi.org/10.1109/ICSE48619.2023.00022

Debenedetti E, Zhang J, Balunović M, et al (2024) AgentDojo: A dynamic environment to evaluate prompt injection attacks and defenses for LLM agents. In: Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track, pp 82895–82920, https://doi.org/10.52202/079017-2636

Debenedetti E, Shumailov I, Fan T, et al (2025) Defeating prompt injections by design. https://doi.org/10.48550/arXiv.2503.18813, arXiv:2503.18813

Feinstein AR, Cicchetti DV (1990) High agreement but low kappa: I. the problems of two paradoxes. Journal of Clinical Epidemiology 43(6):543–549. https://doi.org/10.1016/0895-4356(90)90158-L

Fleiss JL (1971) Measuring nominal scale agreement among many raters. Psychological Bulletin 76(5):378–382. https://doi.org/10.1037/h0031619

GitHub (2026a) Centralize sanitization of untrusted GitHub response fields. https://github.com/github/github-mcp-server/pull/3114, Pull request #3114, merged 19 August 2026. Accessed 31 October 2026

GitHub (2026b) Ineligible submissions (GitHub bug bounty). https://bounty.github.com/ineligible, accessed 31 October 2026

Greshake K, Abdelnabi S, Mishra S, et al (2023) Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection.

In: Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security (AISec '23). ACM, pp 79–90, https://doi.org/10.1145/3605764.3623985

Gwet KL (2008) Computing inter-rater reliability and its variance in the presence of high agreement. British Journal of Mathematical and Statistical Psychology 61(1):29–48. https://doi.org/10.1348/000711006X126600

Hasan MM, Li H, Fallahzadeh E, et al (2026) Model context protocol (MCP) at first glance: Studying the security and maintainability of MCP servers. ACM Transactions on Software Engineering and Methodology https://doi.org/10.1145/3814959, just accepted; arXiv:2506.13538

Hines K, Lopez G, Hall M, et al (2024) Defending against indirect prompt injection attacks with spotlighting. https://doi.org/10.48550/arXiv.2403.14720, arXiv:2403.14720

Hou X, Zhao Y, Wang S, et al (2025) Model context protocol (MCP): Landscape, security threats, and future research directions. https://doi.org/10.48550/arXiv.2503.23278, arXiv:2503.23278

Huang C, Huang X, Tran NP, et al (2026) Model Context Protocol threat modeling and analyzing vulnerabilities to prompt injection with tool poisoning. https://doi.org/10.48550/arXiv.2603.22489, arXiv:2603.22489

Ladisa P, Plate H, Martinez M, et al (2023) SoK: Taxonomy of attacks on open-source software supply chains. In: 2023 IEEE Symposium on Security and Privacy (SP). IEEE, pp 1509–1526, https://doi.org/10.1109/SP46215.2023.10179304

Landis JR, Koch GG (1977) The measurement of observer agreement for categorical data. Biometrics 33(1):159–174. https://doi.org/10.2307/2529310

Lin E, Koishybayev I, Dunlap T, et al (2024) UntrustIDE: Exploiting weaknesses in VS Code extensions. In: Proceedings of the 2024 Network and Distributed System Security Symposium (NDSS). Internet Society, https://doi.org/10.14722/ndss.2024.24073

Liu S, Tang X, Yang X, et al (2026) When the manual lies: A realistic benchmark to evaluate MCP poisoning attacks for LLM agents. https://doi.org/10.48550/arXiv.2605.24069, arXiv:2605.24069

Liu T, Deng Z, Meng G, et al (2024a) Demystifying RCE vulnerabilities in LLM-integrated apps. In: Proceedings of the 2024 ACM SIGSAC Conference on Computer and Communications Security (CCS '24). ACM, https://doi.org/10.1145/3658644.3690338, arXiv:2309.02926

Liu Y, Jia Y, Geng R, et al (2024b) Formalizing and benchmarking prompt injection attacks and defenses. In: 33rd USENIX Security Symposium (USENIX Security

24). USENIX Association, pp 1831–1847, URL https://www.usenix.org/conference/usenixsecurity24/presentation/liu-yupei

Liu Y, Zhao Y, Lyu Y, et al (2025) "Your AI, My Shell": Demystifying prompt injection attacks on agentic AI coding editors. https://doi.org/10.48550/arXiv.2509.22040, arXiv:2509.22040

Luz E (2026) PromptFiction: A one-click flaw that made Claude Desktop act without consent. https://www.oasis.security/blog/claude-desktop-vulnerability, Oasis Security blog, 28 July 2026. Accessed 31 October 2026

Microsoft (2026) MCP configuration reference (Visual Studio Code documentation). https://code.visualstudio.com/docs/agents/reference/mcp-configuration, accessed 31 October 2026

Model Context Protocol (2025) SEP-1766: Digest-pinned tool versioning and interceptor-based validation in MCP. https://github.com/modelcontextprotocol/modelcontextprotocol/issues/1766, GitHub issue, opened 5 November 2025, closed 24 June 2026. Accessed 31 October 2026

Model Context Protocol (2026a) Specification, revision 2026-07-28. https://modelcontextprotocol.io/specification/2026-07-28, accessed 31 October 2026

Model Context Protocol (2026b) Specification, revision 2026-07-28: Server utilities — caching. https://modelcontextprotocol.io/specification/2026-07-28/server/utilities/caching, accessed 27 October 2026

Nasr M, Carlini N, Sitawarin C, et al (2025) The attacker moves second: Stronger adaptive attacks bypass defenses against LLM jailbreaks and prompt injections. https://doi.org/10.48550/arXiv.2510.09023, arXiv:2510.09023

Ohm M, Plate H, Sykosch A, et al (2020) Backstabber's knife collection: A review of open source software supply chain attacks. In: Maurice C, Bilge L, Stringhini G, et al (eds) Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA 2020), Lecture Notes in Computer Science, vol 12223. Springer, pp 23–43, https://doi.org/10.1007/978-3-030-52683-2_2

OpenAI (2026) Configuration reference (Codex documentation). https://developers.openai.com/codex/config-reference, accessed 17 October 2026

Qwen Team (2026) Settings (Qwen Code documentation). https://github.com/QwenLM/qwen-code/blob/main/docs/users/configuration/settings.md, accessed 17 October 2026

Radosevich B, Halloran J (2025) MCP safety audit: LLMs with the Model Context Protocol allow major security exploits. https://doi.org/10.48550/arXiv.2504.03767, arXiv:2504.03767

Rosales F (2026) GitSpawn: A single flaw lets untrusted repos run code in Claude Code, Codex, Cursor, and Grok. https://www.manifold.security/blog/ai-coding-agents-git-hijack, Manifold Security blog, 1 September 2026. Accessed 31 October 2026

Ruflo maintainers (2026) CVE-2026-59726 (GHSA-c4hm-4h84-2cf3): Unauthenticated RCE in MCP bridge default docker-compose deployment. https://www.cve.org/CVERecord?id=CVE-2026-59726, accessed 31 October 2026

Sentry (2026) Expose event provenance and action gates in MCP tool results. https://github.com/getsentry/toolkit/issues/1093, GitHub issue #1093, opened 15 June 2026. Accessed 31 October 2026

Song H, Shen Y, Luo W, et al (2025) Beyond the protocol: Unveiling attack vectors in the Model Context Protocol (MCP) ecosystem. https://doi.org/10.48550/arXiv.2506.02040, arXiv:2506.02040

Sun P, Kang Z, Jin Q, et al (2026) VIPER-MCP: Detecting and exploiting taint-style vulnerabilities in Model Context Protocol servers. https://doi.org/10.48550/arXiv.2605.21392, arXiv:2605.21392

Toeppe B, Barrak A, Ksontini E (2026) A large-scale dataset of MCP implementations on GitHub. In: Proceedings of the 23rd International Conference on Mining Software Repositories (MSR '26). ACM, pp 610–614, https://doi.org/10.1145/3793302.3793311, arXiv:2607.10123

Utevsky R (2026a) The Cursor deeplink vulnerability that turns a "review this PR" click into remote code execution (DeepJack). https://adversa.ai/blog/cursor-security-deepjack-deeplink-vulnerability-mcp-rce/, Adversa AI blog, 15 July 2026. Accessed 31 October 2026

Utevsky R (2026b) TrustFall: Coding agent security flaw enables one-click RCE in Claude, Cursor, Gemini CLI and GitHub Copilot. https://adversa.ai/blog/trustfall-coding-agent-security-flaw-rce-claude-cursor-gemini-cli-copilot/, Adversa AI blog, 7 May 2026. Accessed 31 October 2026

Wang R, Guo J, Gao C, et al (2025) Can LLMs replace human evaluators? an empirical study of LLM-as-a-judge in software engineering. Proceedings of the ACM on Software Engineering 2(ISSTA):1955–1977. https://doi.org/10.1145/3728963

Wang Z, Gao Y, Wang Y, et al (2026) MCPTox: A benchmark for tool poisoning on real-world MCP servers. In: Proceedings of the AAAI Conference on Artificial Intelligence, pp 35811–35819, https://doi.org/10.1609/aaai.v40i42.40895, arXiv:2508.14925

Wu J, Wang H, Zhang Y, et al (2026) MOSAIC: Knowledge-guided CLI command composition attack in LLM coding agents. https://doi.org/10.48550/arXiv.2607.

02857, arXiv:2607.02857

Yan L, Li R, Han X, et al (2026) Trust no tool: Evaluating and defending LLM agents under untrusted tool feedback. https://doi.org/10.48550/arXiv.2605.17453, arXiv:2605.17453

Yang Y, Gao C, Wu D, et al (2025) MCPSecBench: A systematic security benchmark and playground for testing Model Context Protocols. https://doi.org/10.48550/arXiv.2508.13220, arXiv:2508.13220

Zerouali A, Mens T, Decan A, et al (2022) On the impact of security vulnerabilities in the npm and RubyGems dependency networks. Empirical Software Engineering 27(5):107. https://doi.org/10.1007/s10664-022-10154-1

Zhan Q, Liang Z, Ying Z, et al (2024) InjecAgent: Benchmarking indirect prompt injections in tool-integrated large language model agents. In: Findings of the Association for Computational Linguistics: ACL 2024, pp 10471–10506, https://doi.org/10.18653/v1/2024.findings-acl.624

Zheng Y, Hu Y, Yu T, et al (2025) AgentSight: System-level observability for AI agents using eBPF. In: Proceedings of the 4th Workshop on Practical Adoption Challenges of ML for Systems (PACMI '25). ACM, pp 110–115, https://doi.org/10.1145/3766882.3767169

Zhu F, Zhao B, Lin R, et al (2026) Beyond the payload: How user invocation shapes coding agent vulnerability to repository poisoning. https://doi.org/10.48550/arXiv.2608.30686, arXiv:2608.30686

Zimmermann M, Staicu CA, Tenny C, et al (2019) Small world with high risks: A study of security threats in the npm ecosystem. In: 28th USENIX Security Symposium (USENIX Security 19). USENIX Association, pp 995–1010, URL https://www.usenix.org/conference/usenixsecurity19/presentation/zimmerman

Appendix A Classification criteria (frozen)

The text below is the system prompt given to every model rater, reproduced verbatim from the constant `CRITERIA` of `classify.py`. The same text, translated into Italian under the rules of D46, is the criteria text on the sheet of the human rater R0.

- Criteria version: `v3 (preregistration v2 + bundle sentence of D41 point 1, 25 Oct 2026)`.
- SHA-256 of the exact string (UTF-8, line breaks as in the source file): `1dbc3906da9a23e7b2b571c9b58214dc773ca655a997e50c159f789cab8700b8`. The same hash is recorded in the manifest of the RQ1 confirmatory run.

  You are classifying a public security disclosure about an AI agent or one of its connectors. Assign exactly one class by trigger point:

1. Before trust - runs at startup or before the user's trust decision. Includes flaws in the agent's own startup and execution machinery (sandbox boundaries, background tool invocations, path resolution) that let repository content run code without any user or model decision.
2. Interaction layer - the approval or confirmation step misrepresents or skips what happens.
3. Plumbing - classic authentication, authorization or origin flaws in the services around the agent: MCP servers, SDKs, connectors, bridges.
4. Authority - the model is manipulated; the impact is set by the privileges it holds.

Decision rule: if the attack only works because the model follows instructions it should not have trusted, the case is class 4, whatever the delivery channel. Otherwise, the case goes to the earliest step that fails without any model decision: before the trust decision (1), in what the user is shown or asked to approve (2), or in the authentication and authorization of the services around the agent (3).
Counting rule: one disclosure is one case, regardless of how many findings it bundles. If the disclosure bundles several findings, apply the decision rule across them: if at least one finding works without the model following instructions it should not have trusted, give the earliest of classes 1-3 among those findings; give class 4 only if every finding needs the model to follow such instructions.
Answer with one JSON object only: {"class": <1|2|3|4>, "confidence": <0-1>, "reason": "<one sentence>"}

Relation to the preregistration.

The four class definitions and the decision rule are those of section 2 of preregistration v2 (frozen 11 October 2026), where they read:

Four classes by trigger point:

1. **Before trust** — runs at startup or before the user's trust decision. *Includes flaws in the agent's own startup and execution machinery (sandbox boundaries, background tool invocations, path resolution) that let repository content run code without any user or model decision.*
2. **Interaction layer** — the approval or confirmation step misrepresents or skips what happens.
3. **Plumbing** — classic authentication, authorization or origin flaws in the services around the agent: MCP servers, SDKs, connectors, bridges.
4. **Authority** — the model is manipulated; the impact is set by the privileges it holds.

Decision rule: *If the attack only works because the model follows instructions it should not have trusted, the case is class 4, whatever the delivery channel. Otherwise, the case goes to the earliest step that fails without any model decision: before the trust decision (1), in what the user is shown or asked to approve (2), or in the authentication and authorization of the services around the agent (3).*

Counting rule: one disclosure = one case, regardless of how many findings it bundles.

The classifier text differs from section 2 only in form and in one added sentence: plain text in place of bold, italics and dashes; "is" in place of "=" in the counting rule;

an opening instruction and a closing answer format for the model; and the bundle sentence of the counting rule ("If the disclosure bundles several findings, ..."), added by deviation D41 point 1 on 25 October 2026, before any RQ1 confirmatory run. The wording of the four classes and of the decision rule is unchanged. In section 2, the italic sentence of class 1 and the scope sentence of class 3 were added after the pilot and before the freeze (preregistration section 7).

Appendix B Exact prompts shown by each client

The text that each client put on screen at the questions discussed in Sections 6 and 7, with the options in the order shown, the pre-selected option and the SHA-256 of each capture, is in Online Resource 1, Sect. B (formerly this appendix; unchanged).

Appendix C Disclosure log

This study, and the blog series in which its lab results were first published, follows one rule: a new security-relevant observation about a product goes to its vendor before it is discussed in the blog series or in this paper, and until it is fixed or the vendor agrees to publication the text reports at most its class and status, and only as far as the vendor's disclosure terms allow; a behaviour the vendor already documents is not reported but listed as documented. A behaviour that is merely visible in public source code is not treated as already known: each such case was triaged before choosing a channel. Table C1 lists the public or authorised entries.

Appendix D Deviations from the preregistration

Every deviation from preregistration v2 (D1–D56), in the order of the public log `DEVIATIONS.md`, with its date, the research question affected and which data existed when it was written, is listed in Online Resource 1, Sect. D (formerly this appendix; unchanged); the full text of every entry is in the log. Table 1 gives the decisions that could affect a confirmatory verdict.

Table C1 Disclosure log (public or authorised entries). "Documented": the vendor's own documentation or a public change describes the behaviour, so it is listed as documented rather than reported.

Observed behaviour	Product and version	Status
RQ2 (repository-declared MCP servers), measured 28 September and 9 October 2026		
Non-interactive mode starts the servers of `.mcp.json` without a question, also in never-trusted folders	Claude Code 2.1.283	documented by the vendor (Anthropic 2026c); cited, not reported
The pre-selected answer to the trust question is the one that trusts; trusting starts the declared servers	Gemini CLI, Codex CLI, Cursor CLI	design choices visible on screen (Appendix B); not reported
No folder-trust question; a per-server approval shows the full command, and its pre-selected answer approves; approving starts the server	Qwen Code 0.24.6	design choice visible on screen (Appendix B); not reported
With the chat open, starts the workspace server without a question when a tool is needed	VS Code 1.139.1 + Copilot	documented (workspace MCP servers inherit Workspace Trust (Microsoft 2026)); not reported
Project server disabled until the user switches it on; the command is not shown	Cursor IDE 3.22.7	cautious default; the missing command is a usability point; not reported
RQ3 (approved tool changes), matrix of 15 October 2026, triage of 17 October 2026		
Re-reads the tool list on `list_changed` and uses the new definition without showing anything; rules by name	Claude Code 2.1.300	documented; not reported
Notification received and ignored; approval bound to server and tool	Codex CLI 0.162.1	partly documented; not reported
Notification received and ignored	Cursor CLI 2026.10.14	not reported
Shows "Tools updated for server" and continues without asking	Gemini CLI 0.64.0	documented; not reported
Does not subscribe to notifications; default mode without approvals	Goose 1.54.0	partly documented; not reported
Notification received and ignored; rules by name	Qwen Code 0.25.1	partly documented; not reported
Re-reads the list on `list_changed`; an "Always Allow" survives the change	VS Code 1.139.1 + Copilot	documented; not reported
Uses the changed definition without showing anything; first call without a question	Cursor IDE 3.22.7	partly documented; not reported
RQ4 (untrusted-content defences in official MCP servers), source read on 9 October 2026		
Tools whose third-party text does not pass through the server's own defence	AWS Labs MCP servers (IaC, SQL Server, Oracle)	"AWS was informed before publication and assessed these gaps as having no significant security impact."
Third-party data returned without provenance marking	Sentry MCP	already public (open issue (Sentry 2026)); no report
File, diff and log content returned unsanitised (only an optional provenance label; the file and log tools are coded partial in Table 8)	GitHub MCP server	files and diffs: a declared public design choice (pull request of 19 August 2026 (GitHub 2026a)); logs: not covered by that statement, triaged (22 October 2026) as content steering the model without an authorisation impact, which the vendor's bounty policy does not treat as eligible (GitHub 2026b); no report